

视差螺旋桨 2

Spin2 语言文档

2021-03-23

v35L

文档状态

版本	日期	进步
	2020_02_06	已开始文件。
v34t	2020_07_15	已添加 DEBUG,文档已更新。
v34u	2020_07_19	DEBUG 已改进,文档已更新。
第 35 节	2020_11_18	DEBUG 经过改进,具有抗锯齿功能,添加了 QSIN / QCOS,文档保持最新。
v35e	2021_01_06	添加了 DEBUG_BAUD 符号。已修复 Spin2 堆栈定位错误。
v35f	2021_01_29	已修复 DEBUG。之前在 63 个 DEBUG 处出错,现在错误到 255 个。之前并不总是重置 DEBUG.log 文件。
v35克	2021_02_13	调试修复。行剪辑例程会导致浮点异常和内存访问冲突。
v35小时	2021-02-15	· Spin2 解释器中的前 16 个 LUT 寄存器被释放,以允许流式传输器 “imm-->LUT”的使用。这是旨在通过中断在解释器运行的同一个 cog 中支持 1/2/4 位视频。为了进行补偿,内联 PASM 限额从 134 美元降至 124 美元。 · 添加了新的 DEBUG_WINDOWS_OFF 符号,以禁止在以下情况下打开任何 DEBUG 窗口：下载。现在可以设置 DEBUG_BAUD 来改变 DEBUG 与 PNut.exe 一起使用的波特率。
v35i	2021-02-20	· 增加了命令行仅 DEBUG 模式,用于呈现闪存编程的 DEBUG 数据和显示。 · 修复SCOPE_XY 中的浮点错误。
v35j	2021-03-16	修复了 DEBUG_BAUD <> 2_000_000 在某些主板上不起作用的问题。
v35k	2021-03-19	将 DOWNLOAD_BAUD 添加到现有的 DEBUG_BAUD,以覆盖默认的 2 Mbaud 下载和 DEBUG。
v35L	2021-03-23	为 PNut.exe 添加了完整的命令行界面,并包含用于调用 PNut.exe 和返回错误的批处理文件状态到 STDOUT,STDERR 和 ERRORLEVEL。请参阅“PNut.exe 的命令行选项”。

概述

Spin2 语言的设计非常简单,但功能强大。Spin2 并不隐藏使计算机工作的底层二进制现象,而是允许您利用它来有效地编程。Spin2 还支持汇编语言,既可以作为内联序列,也可以作为独立程序。

有编程经验的人可以在很短的时间内对 Spin2 有扎实的理解。学习 Spin2 的好处是可以让你专注于你的想法,无需处理大量的类型转换和使用规则。您的大脑会很高兴保持忙碌,编译+下载+执行时间不到 1 秒。

在 Spin2 中：

- 没有变量类型,只有三种字大小:BYTE (8 位)、WORD (16 位)和 LONG (32 位),每种类型都支持位字段。
 - 所有数学运算均以 32 位执行,并且存在有符号和无符号运算符,以区别重要。
- 程序,称为对象,可以轻松合并其他作者编写的许多其他对象,而无需预先了解您的项目。
- 对象编译为紧凑的、硬件加速的字节码块,从而触发驻留在 cog 中的解释器代码的短序列。
- 代码不区分大小写,因此您可以根据需要将代码大写。
- 符号名称的长度最多可达 32 个字符。

在本文档中,为了清晰起见,所有关键字均采用大写,小写字母代表用户定义的符号名称。

项目结构

Spin2 程序由一个或多个对象构建而成。对象是包含至少一个公共方法的文件,以及可选常量、子对象、变量、附加方法和数据。对象被组装成一个顶层对象,内部有子对象层次结构。每个对象实例在运行时都会获得一组由对象定义的变量,以保持其独特的运行状态。

对象的不同部分在块内声明,所有块都以 3 个字母的块标识符开头。

编译器实际上可以生成仅 PASM 的程序,以及 Spin2+PASM 程序,具体取决于 .spin2 文件中存在哪些块。

堵塞 标识符	区块内容	Spin2+PASM 程序	仅限 PASM 程序
反对	常量声明 (CON 是初始/默认块类型)	允许	允许
对象	子对象实例化	允许	不允许
风险前缀	变量声明	允许	不允许
公共事务局	供父对象和本对象使用的公共方法	必需的	不允许
优先等级	在此对象内使用的私有方法	允许	不允许
数据保护	数据声明,包括 PASM 代码	允许	必需的

以下是一些最小的 Spin2 和仅 PASM 程序。如果您将它们复制并粘贴到 PNut.exe 中,您可以按 F10 来运行它们。

最小 Spin2 程序	PUB MinimalSpin2Program() 第一个 PUB 方法执行 重复 PINWRITE (63..56,GETRND ()) 等待ITMS(100) 将随机模式写入 P63..P56 等待十分之一秒,循环
最小 块级常量块级常量 程序	数据保护 组织机构 在中心 \$00000 处启动 PASM,用于齿轮 \$000 环形 DRVRND #56 附加组件 7 WAITX ##clkfreq_/10 #loop 跳转 将随机模式写入 P63..P56 等待十分之一秒,循环

这是一个包含每种块类型的 Spin2 程序。

全块 Spin2 程序	CON _clkfreq = 297_000_000 设置时钟频率 OBJ vga: “VGA_640x480_文本_80x40” 实例化 vga 对象 VAR 时间,i 声明对象范围的变量 公共去 () 第一个公共方法执行后,cog 停止 vga.启动(8) 在基座引脚 8 上启动 vga 发送 :=@vga.print 建立 SEND 指针 发送 (4, \$004040,5, \$00FFFF) 将浅青色设置为深青色 时间 := GETCT() 捕获时间 我:= @文本 重复@textend-i 发送 (字节[i++]) 打印文件到 vga 屏幕 时间 := GETCT() - 时间 以时钟周期为单位捕获时间增量 时间:= MULDIV64 (时间,1_000_000,clkfreq) 获取微秒内的时间增量 SEND(12, 打印耗时 ,dec(time), 微秒。) 打印时间增量 PRI dec(值) 标志,位置,数字 私有方法打印小数,三个局部变量 标志~ 重置数字打印标志 位置 := 1_000_000_000 从十亿开始向下 重复 如果标志 != (数字 := 值 / 位置 // 10) 位置 == 1 打印一个数字? 发送(“0” +数字) 是的 如果 LOOKDOWN (地点:1_000_000_000,1_000_000,1_000) 还打印逗号吗? 发送 (” , ”) 是的 WHILE 位置 /= 10 “下一个地方,完成了吗?” 数据保护 文本文件 “VGA_640x480_text_80x40.txt”文本结束 包括用于打印的原始文件数据
-------------------	--

以下是每种块类型的细分。

CON 块

CON 块用于定义可在整个文件中使用的符号常量。

	字 e,f[5],g,h[10]	逗号分隔的声明
	LONG 值	长变量
	LONG 值[15]	长变量数组 (15 个长整型)
	长整型[100],j,k,l	逗号分隔的声明
	对齐	字对齐到集线器内存,根据需要推进变量指针
	对齐	长对齐到集线器内存,根据需要推进变量指针
	BYTE 位图[640*480]	..对于制作用于 FIFO 包装的长对齐缓冲区很有用

PUB 和 PRI 块

PUB 和 PRI 块用于定义公共和私有可执行 Spin2 方法。

- PUB 方法可供父对象以及定义它们的对象使用。
- PRI 方法仅对定义它们的对象可用。
- 当对象作为顶级对象运行时,将执行对象中的第一个 PUB 方法。
- 方法可以有 0 到 127 个输入参数,所有输入参数都是单个长整型。
- 方法可以有 0 到 15 个输出结果,所有输出结果都是单个长整型。
- 方法最多可以有 64KB 的局部变量,可以是字节、字和长整型（默认）,以单精度和数组形式存在。
 - 局部变量大小覆盖（字节/字）仅适用于正在声明的变量,而不适用于后续变量。
- 在方法进入时结果初始化为零,而局部变量未定义。
- 参数、结果、局部变量按照声明的顺序被打包到堆栈内存中。
 - 在线 PASM 代码可以通过具有相同符号名的寄存器访问前 16 个长参数...结果...本地变量。

PUB/PRI 语法如下：

PUB/PRI 方法名称({参数{,...}}){: 结果{,...}}{| {ALIGNW/ALIGNL}{ BYTE/WORD/LONG} localvar[{count}]{,...}}

PUB / PRI 声明 (方法代码将位于每个声明下方)	输入 参数 (长)	输出 结果 (长)	当地的 变量 (长整型、字、字节)
公共去 ()	0	0	0
PUB 设置ADC(引脚)	1	0	0
PUB StartTx (引脚,波特率) :好的	2	1	0
PRI 旋转XY (X,Y,角度) :NewX,NewY p,q,r	3	2	3 条长线
PRI 洗牌() i,j	0	0	2 条长线
PRI FFT1024(数据指针) a,b,x[1024],y[1024]	1	0	1+1+1024+1024 长整型
PRI ReMix() : 长度,采样率 WORD Buff[20000],k	0	2	20000 字 + 1 篇长文
PRI StrCheck(StrPtrA, StrPtrB) :通过 俄,字节Str[64]	2	1	1 个长整型 + 64 个字节

DAT 块

DAT块用于表达数据和PASM代码。

- 数据按照声明的顺序打包到内存中,从长对齐的地址开始。
- 数据使用以下语法表示:{symbolName} BYTE/WORD/LONG data[{count}]{,data...}
- 数据和 PASM 指令之前的符号解析为地址
 - 在 Spin2+PASM 程序中,集线器地址相对于对象的起始位置,可以按如下方式引用：
 - SymbolName 将根据符号的大小（字节/字/长）返回符号的数据。
 - @SymbolName 将返回数据的地址。
 - '@@SymbolName' 将数据中的 '@Symbol' 转换为绝对地址（参见“DAT 数据指针”）
 - 在仅限 PASM 的程序中,集线器地址是绝对的。

数据保护			
符号和数据			
数据保护			没有数据的符号采用前一个声明的大小
HexChrs 符号0	字节	“0123456789ABCDEF”	HexChrs 是指向 “0”的字节符号 symbol0 是指向 “F”之后的字节符号
形态符号1	单词	\$CCCC,\$3333,\$AAAA,\$5555	模式是指向 \$CCCC 的文字符号 symbol1 是指向 \$55555 之后的单词符号
十亿符号2	长整型	1_000_000_000	“十亿”是一个指向 1_000_000_000 的长符号 symbol2 是一个指向 1_000_000_000 之后的长符号
DoNothing 符号3	不适用		DoNothing 是一个指向 NOP 指令的长符号 symbol3 是一个指向 NOP 指令后的长符号
符号4 符号5 符号6	字节 单词 长的		symbol4 是一个指向 \$78 的字节符号 symbol5 是一个指向 \$5678 的单词符号 symbol6 是一个指向 \$12345678 的长符号
	长	\$12345678	长值 \$12345678
	字节	100[64]	64 个字节,值为 100
	字节 字节	10,WORD 500,LONG \$FC000 恒等无功功率 99、恒等无功功率 -99	BYTE/WORD/LONG 允许单个值覆盖 允许 FVAR/FVARS 覆盖,可通过 RFVAR/RFVARS 读取
文件数据	文件	“文件名”	包含二进制文件,FileDat是指向文件的字节符号
	对齐 对齐		如果需要,通过发出零字节将字对齐到集线器 如果必要的话,通过发射 1 到 3 个零字节与集线器进行长对齐

数据保护			
数据指针			
数据保护			
字符串0	字节	“猴子” ,0 带符号的字符串	
字符串1	字节	“大猩猩” ,0	
Str2	字节	“黑猩猩” ,0	
Str3	字节	“人猿” ,0	
字符串列表	单词	@Str0	在 Spin2 中,这些是字符串相对于对象起始处的偏移量
	单词	@Str1	在 Spin2 中,@@StrList[i] 将返回 i = 0..3 的 Str0..Str3 的地址
	单词	@Str2	在仅 PASM 程序中,这些是字符串的绝对地址
	单词	@Str3	(使用 WORD 假设偏移量/地址低于 64KB)

数据保护			
执行总监			
数据保护	组织机构		开始一个 cog-exec 程序 (ORG 之前不允许有符号)
IncPins	在寄存器中设置寄存器	DIRA,#\$FF	COGINIT(16, @IncPins, 0) 将在免费 cog 中启动此程序
	添加	出塔, #1	对于 Spin2 代码,IncPins 是 MOV 指令 (长)
	跳转	#环形	对于 Spin2 代码,@IncPins 是 MOV 指令的集线器地址
			到 PASM 代码,Loop 是 ADD 指令的 cog 地址 (\$001)
	组织机构		设置 cog-exec 模式,cog 地址 = \$000,cog 限制 = \$1F8 (reg,均为默认值)
	组织机构	\$100	设置 cog-exec 模式,cog 地址 = \$100,cog 限制 = \$1F8 (reg,默认限制)
	组织机构	\$120,\$140 \$200	设置 cog-exec 模式,cog 地址 = \$120,cog 限制 = \$140 (reg)
	组织机构		设置 cog-exec 模式,cog 地址 = \$200,cog 限制 = \$400 (LUT,默认限制)
	组织机构	\$300,\$380	设置 cog-exec 模式,cog 地址 = \$300,cog 限制 = \$380 (LUT)
	添加	注册, #1	在 cog-exec 模式下,指令强制对齐 cog/LUT 寄存器
	有机生长因子	\$040	用零填充 cog 地址 \$040 (ORGF 之前不允许有符号)
	合身	\$020	测试以确保 cog 地址不超过 \$020
十是是增益	可再生资源	1	保留 1 个寄存器,将 cog 地址前进 1,不前进 hub 地址
	可再生资源	1	保留 1 个寄存器,将 cog 地址前进 1,不前进 hub 地址
	可再生资源		保留 1 个寄存器,将 cog 地址前进 1,不前进 hub 地址
	可再生资源	1 16	保留 16 个寄存器,将 cog 地址提前 16,不提前 hub 地址

数据保护 中心执行程序			
数据保护	奥格		启动一个 hub-exec 程序（ORGH 之前不允许有符号）
IncPins 环形	在寄存器中设置寄存器	DIRA,#\$FF	COGINIT(32+16, @IncPins, 0) 将在免费 cog 中启动此程序
	添加	出塔,#1	在 Spin2 中,IncPins 是 MOV 指令（长）
	跳转	#环形	在 Spin2 中,@IncPins 是 MOV 指令的集线器地址
			在 PASM 中,Loop 是 ADD 指令的集线器地址 (\$00404)
	奥格		设置 hub-exec 模式,hub 原点 = \$00400,原点限制 = \$100000（均为默认值）
	奥格斯特 1000 美元		设置 hub-exec 模式,hub 原点 = \$01000,原点限制 = \$100000（默认限制）
	奥格	\$FC000,\$FC800	设置 hub-exec 模式,hub 原点 = \$FC000,原点限制 = \$FC800
	合身	2000美元	测试以确保中心地址不超过 2000 美元

在 hub-exec 代码方面,Spin2+PASM 程序和仅 PASM 程序之间存在一些差异：

Spin2+PASM 程序	· Hub-exec 代码必须使用相对寻址,因为它并不位于其原点。 · LOC 指令可用于获取相关 hub-exec 代码中数据资产的地址。 · ORGH 必须指定至少\$400,以便汇编纯 hub-exec 代码。 · 默认的 ORGH 地址 \$400 始终是合适的,除非您正在编写代码,在运行时移动到其实际的 ORGH 地址,以便它可以使用绝对寻址。		
	数据保护	奥格	设置 hub-exec 模式并将原点设置为 \$400
		奥格	\$FC000 设置 hub-exec 模式并将原点设置为 \$FC000
仅限 PASM 程序	· Hub-exec 代码可能使用绝对寻址和相对寻址,因为原点总是与集线器地址匹配。 · ORGH 用零填充集线器内存,直到指定地址。		
	数据保护	奥格	在当前集线器地址设置集线器执行模式
		奥格	\$400 设置 hub-exec 模式并用零填充 hub 内存至 \$400

Spin2 语言

常量

常量解析为 32 位值,可以表示如下：

常量	示例	描述
十进制	1	· 十进制值使用数字 “0” .. “9” · 第一个数字后允许使用下划线 “_”作为占位符
	-150	
	3_000_000	
十六进制\$10B	\$AA55	· 十六进制值以 “\$”开头,使用数字 “0” .. “9”和 “A” .. “F” · 第一个数字后允许使用下划线 “_”作为占位符
	\$FFFF_FFFF	

双二进制%21 %01_23 % %3333_2222_1111_0000	· 双二进制值以 %% 开头,使用数字 0 .. 3 · 第一个数字后允许使用下划线 _ 作为占位符
二进制 %0110 %1_1111_1000 %0001_0010_0011_0100	· 二进制值以 % 开头,使用数字 0 和 1 · 第一个数字后允许使用下划线 _ 作为占位符
漂浮 -1.0 1_250_000.0 1e9 -1.23456e-7	· 浮点值使用数字 “0” .. “9” ,其中包含 “.”和/或 “e”· 浮点数以 IEEE-754 单精度 32 位格式编码 · 第一个数字后允许使用下划线 “_”作为占位符· 浮点数不是 Spin2 的一部分,但库可以提供浮点函数
特点	“H” · 引号中的单个字符解析为 7 位 ASCII 值

变量

在Spin2中,既有用户定义变量,也有永久变量。用户定义变量来源如下表所示,永久变量如表所示。

- VAR 变量（集线器）
 - PUB/PRI 参数、返回值和局部变量（集线器）
 - DAT 符号（集线器）
 - Cog 寄存器

变量 (全部很长)	多变的 姓名	地址或偏移量	描述	有用 Spin2	有用 Spin2-PASM	有用 仅限 PASM
枢纽位置	时钟模式 时钟频率	\$00040\$00044	时钟模式值 时钟频率值	是的 是的	是的 是的	不 不
枢纽增值转售商	变量基础	+0	对象基指针 ,@VARBASE 是 VAR 基,由方法指针调用使用	或许	不	不
齿轮寄存器	PR0	\$1D8	Spin2 <=> PASM 通信	是的	是的	不
	PR1	\$1D9		是的	是的	不
	PR2	1DA 美元		是的	是的	不
	PR3	\$1DB		是的	是的	不
	PR4	1DC美元		是的	是的	不
	PR5	\$1DD		是的	是的	不
	PR6	1DE美元		是的	是的	不
	PR7	\$1DF		是的	是的	不
	联合治疗3 雷特3	\$1F0 \$1F1	中断 JMP 和 RET	不 不	是的 是的	是的 是的
	联合治疗2 雷特2	\$1F2 \$1F3		不 不	是的 是的	是的 是的
	联合治疗1 雷特1	\$1F4 \$1F5		不 不	是的 是的	是的 是的
	REFADN 铅	\$1F6 \$1F7		不 不	是的 是的	是的 是的
	程序库特定性成员 物理治疗委员会	\$1F8 \$1F9		不 不	是的 是的	是的 是的
	爱尔兰投资局 迪尔比	\$1FA \$1FB	P31..P0 的输出使能 P63..P32 的输出使能 P31..P0 的输出状态 P63..P32 的输出状态 P31..P0 的输入状态 P63..P32 的输入 状态	是的 是的	是的 是的	是的 是的
	欧塔 输出端	\$1FC 1金融时报		是的 是的	是的 是的	是的 是的
	伊纳 吟咏美辛	\$1FE \$1FF		是的 是的	是的 是的	是的 是的

在 Spin2 中,所有变量都可以作为位域进行索引和访问。此外,符号集线器变量可以具有 BYTE/WORD/LONG 大小覆盖：

变量用法	例子	描述
清楚的	AnyVar HubVar.WORD BYTE[地址] REG[寄存器]	集线器或永久寄存器变量 具有 BYTE/WORD/LONG 大小覆盖的集线器变量 根据地址选择 Hub BYTE/WORD/LONG 注册， register 可能是在 ORG 部分中声明的符号
带索引	AnyVar[索引] HubVar.BYTE[索引] LONG[地址][索引] REG[寄存器][索引]	带索引的集线器或永久寄存器变量 具有尺寸覆盖和索引的集线器变量 通过带索引的地址来访问 Hub BYTE/WORD/LONG 使用索引注册
使用 Bitfield	AnyVar.[位域] HubVar.LONG.[位字段] WORD[地址].[位域] REG[寄存器].[位域]	具有位域的集线器或永久寄存器变量 具有大小覆盖和位字段的集线器变量 通过位域地址来连接 Hub BYTE/WORD/LONG 使用位字段进行注册
使用索引和位域AnyVar[索引].[位域]	HubVar.BYTE[索引].[位域] LONG[地址][索引].[位域] REG[寄存器][索引].[位域]	具有索引和位域的集线器或永久寄存器变量 具有大小覆盖、索引和位字段的集线器变量 通过带索引和位域的地址来访问 Hub BYTE/WORD/LONG 使用索引和位域进行注册

位域是一个 10 位值,其中包含位 4..0 中的基数和位 9..5 中的附加位数。位域可以用几种不同的方式定义：

位域	位范围	细节
.[%00000_00000]	0	0 基位 0 以上的附加位,单一位域
.[%00000_11111]	31	0 基位 31 以上的附加位,单一位域

.[%00010_01111]	17..15	基位 15 以上的 2 个附加位,即三位位域
.[%11110_00000]	30..0	基位 0 以上 30 个附加位,即 31 位位域
.[%11111_10000]	15..0, 31..16	基位 16 之上的 31 个附加位,环绕 32 位位域
.[%00001_11111]	0,31	基位 31 上方有 1 个附加位,环绕 2 位位域
[23]	23	仅基本位,不添加任何额外位
.[31..20]	31..20	.[] 中允许使用 Top..Bottom 语法,如果 Top < Bottom 则换行
[5 添加比特 7]	12..5	ADDBITS 可用于计算位域
.[BitfieldCon]	13..9	CON BitfieldCon = 9 ADDBITS 4BitfieldCon 在 PASM 中也很有用
.[位字段变量]	?	BitfieldVar := BaseBit ADDBITS ExtraBits 若 BaseBit + ExtraBits > 31 则换行

除了位域之外,还有引脚域,用于选择同一 32 引脚块内的一系列 I/O 引脚（P63..P32 或 P31..P0）。引脚域是 11 位值,包含位 5..0 表示基本引脚数,位 10..6 表示附加引脚数。引脚区由与引脚接口的指令使用。

平菲尔德	引脚范围	细节
PINLOW(%00000_000000)	0	0 个基极引脚上方的附加引脚,即单引脚引脚区
PINLOW(%00000_111111)	63	0 个基极引脚 63 上方的附加引脚,单引脚引脚区
PINLOW(%00011_100000)	35..32	32 号基座引脚上方有 3 个附加引脚,即四针引脚区
PINLOW(%11111_001000)	7..0, 31..8	基座引脚 8 上方有 31 个附加引脚,环绕 32 引脚引脚区
平洛(19)	19	仅基座引脚,不添加额外引脚
低点(49..40)	49..40	.[] 中允许使用 Top..Bottom 语法,如果 Top < Bottom 则换行
PINLOW(11 添加 4)	15..11	ADDPINS 可用于计算 pinfield
PINLOW (PinfieldCon)	53..50	CON PinfieldCon = 50 ADDPINS 3PinfieldCon 在 PASM 中也很有用
PINLOW(PinfieldVar)	?	PinfieldVar := BasePin ADDPINS ExtraPins 如果 BasePin + ExtraPins > 31 则换行

表达式

- 运行时表达式可以包含常量、变量和方法的返回值
- 编译时表达式只能使用常量。
- 所有表达式都可以使用运算符。

以下是一些表达式的示例：

表达	细节
字节[i++]	‘i’ 指向的字节,后增 ‘i’
(数字 := 值 / 位置 // 10) 或 位置 == 1	带有隐藏 “数字”赋值的布尔值
位置 /= 10	将 “地点”除以 10
“0” +数字	获取 “数字”字符
PIN 读取(17..12)	读取引脚 17..12

运算符

下面是 Spin2 方法中可用的所有运算符的表格。编译时表达式可以使用一元、二元、三元和浮点运算符。

变量前缀 运算符	学期 (仅限方法)	优先事项 (学期)	分配 (仅限方法)	优先事项 (分配)	描述	漂浮 经验值
++ (前)	++变量	1	++变量	1	预增	
-- (前)	--变量	1	--变量	1	预减量	
?? (前)	??变量	1	??变量	1	每 XOR032 进行长迭代,返回伪随机数	
Var-后缀 运算符	学期 (仅限方法)	优先事项 (学期)	分配 (仅限方法)	优先事项 (分配)	描述	漂浮 经验值
(帖子)++	变量++	1	变量++	1	后增	
(邮政) -	变量--	1	变量--	1	后减量	
(邮政) !!	变种!!	1	变种!!	1	后逻辑非 (0 → -1,非 0 → 0)	
(邮政) !	变量!	1	变量!	1	后位非	
(邮政) \	变量\ x	1	变量\ x	1	后分配 x	
(帖子) ~	变种~	1	变种~	1	后清除所有位	
(发帖)~~	变量~~	1	变量~~	1	后置所有位	
地址 运算符	学期 (仅限方法)	优先事项 (学期)			描述	漂浮 经验值
@	@象征	1			VAR/PUB/PRI 变量或 DAT 符号的集线器地址	
@	@方法	1			指向方法的指针,可能是 @object[i]].method	
@@	@@x	1			对象的中心地址 + x, DAT x long @dat_symbol	
#	#reg_symbol	1			cog/LUT DAT 符号的寄存器地址	
一元 运算符	学期	优先事项 (学期)	分配 (仅限方法)	优先事项 (分配)	描述	漂浮 经验值
! ! , 不是	!x	12	!= 变量	1	逻辑非 (0 → -1,非 0 → 0)	
!	!x	2	!= 变量	1	按位非 (1 的补码)	
-	-x	2	= 变量	1	取反 (2 的补码)	
绝对值	ABS 型号	2	ABS=变量	1	绝对值	
编码	编码 x	2	ENCOD= var	1	编码 MSB,0..31	
解码	解码 x	2	解码=var	1	解码,1 << (x & \$1F)	
屏蔽门锁	屏蔽门	2	BMASK= 变量	1	位掩码,(2 << (x & \$1F)) - 1	
—	一个 x	2	—= var	1	将所有 ‘1’ 位相加,0..32	
平方根	平方根 x	2	SQRT= 变量	1	无符号值的平方根	
质量保证	轉識	2	QLOG= 变量	1	无符号值转对数 {5 整数, 27 分数}	
速递	指数函数	2	QEXP=变量	1	对数转为无符号值	
二进制 运算符	学期	优先事项 (学期)	分配 (仅限方法)	优先事项 (分配)	描述	漂浮 经验值
>>	x >> y	3	var >>= y	17	将 x 右移 y 位,插入 0	
<<	x << y	3	变量 <<= y	17	将 x 左移 y 位,插入 0	
特区	x SAR y	3	变量 SAR= y	17	将 x 右移 y 位,插入 MSB	
回流比	x ROR y	3	var ROR= y	17	将 x 右移 y 位	
罗尔	x 方向	3	var ROL= y	17	将 x 左移 y 位	
修订	x 修订 y	3	var REV= y	17	反转 x 的 y LSB 并进行零扩展	
零度	x 零 y	3	var ZEROX= y	17	零扩展至位 y 之上	
信通	x 符号X y	3	var SIGNX= y	17	从 y 位进行符号扩展	
&	x 和 y	4	变量 &= y	17	按位与	
^	+ ^ 是	5	变量 ^= 是	17	按位异或	
	x y	6	变量 = y	17	按位或	
*	x * 是	7	变量 *= 是	17	有符号乘法	
/	x / y	7	变量 /= y	17	有符号除法,返回商	
+/	x +/ y	7	变量 +/= y	17	无符号除法,返回商	
//	x // 7	7	var /= y	17	有符号除法,返回余数	
+/	x +// y	7	变量 +//= y	17	无符号除法,返回余数	
细胞因子	x 坐标	7	变量 SCA= y	17	无符号标度,(x * y) >> 32	
南卡罗来纳大学	x SCAS y	7	var SCAS= y	17	有符号比例,(x * y) >> 30	
压裂	x 分数 y	7	var FRAC= y	17	无符号分数,(x << 32) / y	
+	x + y	8	VAR += y	17	添加	
-	x - y	8	变量 -= y	17	减去	
#>	x #> y	9	var #>= y	17	强制 x => y,有符号	
<#	x <# y	9	var <#= y	17	强制 x <= y,有符号	
添加比特	x 添加位 y	10	var ADDBITS= y	17	创建位域,(x & \$1F) (y & \$1F) << 5	
附加元件	x 添加项 y	10	var ADDPINS= y	17	制作 pinfield,(x & \$3F) (y & \$1F) << 6	
<	x < y	11			有符号小于 (返回 0 或 -1)	
+<	x +< y	11			无符号小于 (返回 0 或 -1)	
<=	x <= y	11			有符号小于或等于 (返回 0 或 -1)	
+<=	x +<= y	11			无符号小于或等于 (返回 0 或 -1)	

==	x == y	11			相等 (返回 0 或 -1)	
<>	x <> y	11			不等于 (返回 0 或 -1)	
>=	x >= y	11			有符号大于或等于 (返回 0 或 -1)	
+>=	x +>= y	11			无符号大于或等于 (返回 0 或 -1)	
>	x > y	11			有符号大于 (返回 0 或 -1)	
+>	x + > y	11			无符号大于 (返回 0 或 -1)	
<=>	x <=> y	11			有符号比较 (<,<=,> 返回 -1,0,1)	
&&, 和	x && y	+三	var &&= y	17	逻辑与 (x <> 0 与 y <> 0,返回 0 或 -1)	
^^,异或	+ ^^ 是	14	变量 ^^= 是	17	逻辑异或 (x <> 0 XOR y <> 0,返回 0 或 -1)	
,或	x y	15	变量 = y	17	逻辑或 (x <> 0 或 y <> 0,返回 0 或 -1)	
三元 操作员	学期	优先事项 (学期)			描述	漂浮 经验值
? :	x?y:z	16			如果 x <> 0 则选择 y,否则选择 z	
分配 操作员			分配 (仅限方法)	优先事项 (分配)	描述	漂浮 经验值
:=			变量 := x v1,v2:= x,y	17	将 var 设置为 x 将 v1 设置为 x,将 v2 设置为 y,等等。(左侧的 _ = 忽略)	
等同 操作员			分配 (仅限 CON 块)	优先事项 (等同)	描述	漂浮 经验值
=			符号 = x	17	在 CON 块中将符号设置为 x	
漂浮 运算符	学期 (仅限常量)				描述	漂浮 经验值
漂浮 ()	浮点 (x)				将整数 x 转换为浮点数	
圆形的 ()	舍入 (x)				将浮点数 x 转换为四舍五入的整数	
截断()	截断(x)				将浮点数 x 转换为截断整数	

内置方法

中心方法	细节
HUBSET(值)	使用 Value 执行 HUBSET 指令
CLKSET (新CLKMODE,新CLKFREQ)	安全地建立新的时钟设置,更新 CLKMODE 和 CLKFREQ
COGSPIN(CogNum,方法 {(Pars)},StkAddr)	在齿轮中启动 Spin2 方法,如果用作表达式元素,则返回齿轮的 ID, -1 = 没有齿轮可用
COGINIT (CogNum,PASMaddr,PTRaValue)	在 cog 中启动 PASM 代码,如果用作表达式元素,则返回 cog 的 ID, -1 = 没有 cog 可用
COGSTOP(CogNum)	停止齿轮 CogNum
COGID() : 认知编号	获取此齿轮的 ID
COGCHK(CogNum) : 正在运行	检查 cog CogNum 是否正在运行,如果正在运行则返回 -1,否则返回 0
LOCKNEW() : 锁号	从库存中取出一个新的 LOCK,如果成功则 LockNum = 0..15,如果没有可用的 LOCK,则 LockNum < 0
LOCKRET(锁定编号)	将特定锁归还到库存中
LOCKTRY (LockNum) :锁状态	尝试捕获某个锁,如果成功则 LockState = -1,如果另一个齿轮捕获了该锁则为 0
LOCKREL(锁定编号)	释放某个锁
LOCKCHK(锁定编号) : 锁定状态	检查某个 LOCK 的状态,LockState[31] = 已捕获,LockState[3:0] = 当前或最后一个所有者 cog
COGATN (CogMask)	根据 16 位 CogMask 选通 ATN 的 cog 输入
POLLATN() : AtnFlag	检查此齿轮是否已收到 ATN 频闪,如果 ATN 频闪,则 AtnFlag = -1,如果没有频闪,则 AtnFlag = 0
等待()	等待此齿轮接收 ATN 频闪

固定方法	细节
PINW PINWRITE (PinField,数据)	使用数据驱动 PinField 引脚
PINL PINLOW (PinField)	将 PinField 引脚驱动至低电平
PINH PINHIGH(PinField)	将 PinField 引脚驱动至高电平
PINT PINTOGGLE (PinField)	驱动并切换 PinField 引脚
PINF PINFLOAT(PinField)	浮动 PinField 引脚
PINR PINREAD(PinField) : PinStates	读取 PinField 引脚
PINSTART(PinField ,模式,Xval,Yval)	启动 PinField 智能引脚:DIR=0,然后 WRPIN=Mode,WXPIN=Xval,WYPIN=Yval,然后 DIR=1
PINCLEAR (PinField)	清除 PinField 智能引脚:DIR=0,然后 WRPIN=0
WRPIN(PinField,数据)	将数据写入 PinField 智能引脚的 ‘模式’寄存器
WXPIN(PinField,数据)	将数据写入 PinField 智能引脚的 ‘X’ 寄存器
WYPIN(PinField,数据)	将数据写入 PinField 智能引脚的 ‘Y’ 寄存器
AKPIN(PinField)	确认 PinField 智能密码
RDPIN(引脚) : Zval	读取 Pin 智能引脚并确认,Zval[31] = 来自 RDPIN 的 C 标志,其他位为 RDPIN 数据
RQPIN(引脚) : Zval	读取 Pin 智能引脚无需确认,Zval[31] = 来自 RQPIN 的 C 标志,其他位是 RQPIN 数据

计时方法	细节
GETCT() :计数	获取32位系统计数器
POLLCT(勾选) : 过去	检查系统计数器是否已超过 “Tick” ,如果已超过则返回 -1,如果未超过则返回 0
等待CT (勾选)	等待系统计数器超过 “Tick”
WAITUS(微秒)	等待微秒,使用 CLKFREQ
WAITMS (毫秒)	等待毫秒,使用 CLKFREQ
GETSEC() :秒	获取自启动以来的秒数,使用 64 位系统计数器和 CLKFREQ,每 136 年滚动一次。
GETMS() :毫秒	获取自启动以来的毫秒数,使用 64 位系统计数器和 CLKFREQ,每 49.7 天翻转一次。

PASM 接口	细节
CALL(注册或中心地址)	在 Addr 处调用 PASM 代码,PASM 代码应避免开寄存器 \$130..\$1D7 和 LUT
REGEXEC(HubAddr)	将 HubAddr 处自定义的 PASM 代码块加载到寄存器中并调用它。请参阅 REGEXEC 描述。
重新加载 (Hub地址)	将 HubAddr 处自定义的 PASM 代码块或数据加载到寄存器中。请参阅 REGLOAD 描述。

数学方法	细节
ROTXY(x,y,angle32bit) : rotx,roty	按角度32位旋转 (x,y)并返回旋转后的 (x,y)
POLXY(长度,角度32位) : x,y	将 (长度,角度32位) 转换为 (x,y)

XYPOL (x,y) :长度,角度32位	将 (x,y) 转换为 (长度,角度32位)
QSIN(长度,角度,2pi) : y	将 (length,0) 旋转 (angle / twopi) * 2Pi 并返回 y。使用 0 表示 twopi = \$1_0000_0000。Twopi 是无符号的。
QCOS(长度,角度,twopi) : x	将 (长度,0) 旋转 (角度 / twopi) * 2Pi 并返回 x。对于 twopi = \$1_0000_0000,使用 0。Twopi 是无符号的。
MULDIV64(mult1,mult2,divisor) :商将 mult1 和 mult2 的 64 位乘积除以 divisor ,返回商 (无符号运算)	
GETRND() : Rnd	获取随机长 (来自 xoroshiro128** PRNG,在启动时使用来自 ADC 的热噪声播种)

记忆方法	细节
GETREGS (HubAddr,CogAddr,计数)	将 CogAddr 处的计数寄存器移动到 HubAddr 处的长整型
SETREGS (HubAddr,CogAddr,计数)	将 HubAddr 处的 Count 长整型移至 CogAddr 处的寄存器
BYTEMOVE(目标,来源,计数)	将 Count 个字节从源移动到目标
WORDMOVE(目标,来源,计数)	将 Count 个单词从源移动到目标
LONGMOVE(目标,来源,计数)	将 Count 长整型从源移动到目标
BYTEFILL(目标,值,计数)	使用值填充目标处的 Count 个字节
WORDFILL(目标,值,计数)	用值填充目标处的 Count 个单词
LONGFILL(目标,值,计数)	在目标处用值填充计数多头

字符串方法	细节
STRSIZE(地址) : 大小	计算 Addr 处以零结尾的字符串的字节数,返回字符串大小,不包括零结尾符
STRCOMP(AddrA,AddrB) :匹配	比较 AddrA 和 AddrB 处的零终止字符串,如果匹配则返回 -1,如果不匹配则返回 0
STRING(“文本” ,9) : 字符串地址	编写一个以零结尾的字符串 (允许带引号的字符和值 1..255) ,返回字符串的地址

索引<=>值方法	细节
LOOKUP (索引:v1,v2..v3 等) :值	使用基于 1 的索引查找值 (允许的值和范围) ,返回值 (如果索引超出范围则返回 0)
LOOKUPZ (索引:v1,v2..v3 等) :值	使用基于 0 的索引查找值 (允许的值和范围) ,返回值 (如果索引超出范围则返回 0)
LOOKDOWN (值:v1,v2..v3等) :索引确定匹配值的基于 1 的索引 (允许的值和范围)	,返回索引 (如果不匹配则为 0)
LOOKDOWNZ (值:v1,v2..v3 等) :索引确定匹配值的基于 0 的索引 (允许的值和范围)	,返回索引 (如果不匹配则为 0)

使用方法

返回单一结果的方法可以用作表达式中的术语：

```
x := GETRND() /100                                获取 0 到 99 之间的随机数

BYTEMOVE(ToStr,FromStr,STRSIZE(FromStr) + 1)
```

返回多个结果的方法（如 POLXY)可用于向其他方法提供多个参数：

```
x,y := SumPoints(POLXY(rho1,theta1), POLXY(rho2,theta2))

...在哪里...
```

```
点数总和 (x1,y1,x2,y2) ~x,y
    返回 x1+x2, y1+y2
```

可以将多个方法的结果分配给变量,或者通过使用下划线代替变量名来忽略它：：

```
x,y := ROTXY(xin,yin,theta) _,y :=          使用 x 和 y 结果
ROTXY(xin,yin,theta) x,_ := ROTXY(xin,yin,theta)  仅使用 y 结果
                                           仅使用 x 结果
```

返回一个或多个结果的用户定义方法也可以用作指令,其中返回值被忽略。但是,返回结果的内置方法（例如 STRSIZE）必须作为表达术语使用。

中止

Spin2 具有“中止”机制,可立即从任何深度的嵌套方法调用返回到在方法名称前使用“\”的基本调用者。单个返回值可以是从中止点传回到基础调用者:

PRI Sub1():错误 错误 := \Sub2() \Sub2()	Sub1 调用 Sub2 并带有 ABORT 陷阱 \ 表示调用方法并捕获任何 ABORT 在这种情况下,ABORT 值被忽略
PRI 子2() Sub3() PINHIGH (0)	Sub2 调用 Sub3 由于 ABORT,Sub3 永远不会返回这里 PINHIGH 永远不会执行
PRI 子3() ABORT错误代码 平洛(0)	Sub3 ABORT,返回 Sub1 并显示 ErrorCode ABORT 并返回 ErrorCode PINLOW 从不执行

不管某个特定方法有多少个返回值,当该方法以前面的“\”调用时,只会有一个返回值,该返回值可能会被忽略。

如果 ABORT 后未指定任何值,则将返回零。

如果调用方法时前面带有“\”,但没有发生 ABORT,则返回零。

如果在调用链中的某处执行 ABORT 而没有“\”陷阱,则 cog 将返回顶层方法并执行 COGSTOP(COGID),从而关闭自身。

中止机制旨在作为一种从出现错误情况的深层嵌套子程序中返回的方法,但它可以用于任何目的。基本上,它是一种返回到基类调用者,而不必在调用链的每一层检查是否符合条件。它会一路返回到调用者,并带有“\”中止陷阱,携带 ABORT 值。您可以组成“\”中止陷阱和 ABORT 点的层次结构。

RECV 与 SEND 类似,是一个特殊的方法指针,它继承自调用方法,并依次传递给所有调用方法。它的目的是为数据。

RECV 可以像方法指针一样被分配,但它必须指向一个不带参数并返回单个值的方法:

接收:= @InMethod

使用 RECV 的示例:

向量自回归

酒吧围棋()
接收 := @GetPattern
重复
PINWRITE(56 ADDPINS 7, !RECV())
等待TMS(125)

PRI GetPattern(): 模式
返回解码(i++ & 7)

在上面的例子中,按重复顺序输出以下值:\$01、\$02、\$04、\$08、\$10、\$20、\$40、\$80 (但对于 LED 则反转)

虽然被调用的方法继承了当前的 RECV 指针,但它可能会出于自己的目的更改它。从该方法返回后,RECV 指针将恢复到该方法之前的状态。被调用。因此,RECV 指针值在方法调用中传播,但不在方法返回中传播。

流量控制

Spin2 有三个基本的流控制结构:

- | | |
|------------------------------|--------------------|
| 如果 / 如果非 + 否则如果 / 否则如果非 + 否则 | - 具有随机决策逻辑的条件执行 |
| 案件/案件_快速 | - 具有单目标和多匹配测试的条件执行 |
| 重复 | - 多种模式的循环执行 |

所有这些构造都使用相对缩进来确定哪些代码属于它们的控制范围:

中频齿轮	如果 cog <> 0
齿轮停止(齿轮-1)	..然后停止 cog
PINCLEAR(av_base_pin_ADDPINS 4)	..然后清除 pin 模式

流控制结构可以按任意顺序嵌套:

```
CASE 标志
0:CASE_FAST 字符
    0:      BYTEFILL(@screen,      , 屏幕尺寸)
        列 := 行 := 0
    1:      列 := 行 := 0
    2..7.标志 := chr
        返回
    8:      如果科尔
        科尔
    9:      重复
        出去 (” ” )
        WHILE 列 & 7
    10:     返回
    11:     颜色 := $00
    12:     颜色 := 80 美元
    13:     换行符()
    其他:输出 (chr)

    2:      col := chr // 列
    3:      row := chr // 行数
    4..7: background0_[标志-$04] := chr << 8
标志 := 0
```

如果 / 如果非 + 否则如果 / 否则如果非 + 否则

IF 结构以 IF 或 IFNOT 开头,并可选地使用 ELSEIF、ELSEIFNOT 和 ELSE。为了成为同一棵决策树的一部分,这些关键字必须具有相同的缩进。

如果 <condition> 不为零,则执行 IF 或 ELSEIF 下的缩进代码。如果 <condition> 为零,则执行 IFNOT 或 ELSEIFNOT 下的代码。如果没有其他条件,则执行 ELSE 下的代码缩进的代码执行:

- | | |
|-------------------------|-------------------------|
| IF / IFNOT <条件> | - 初始 IF 或 IFNOT |
| <缩进代码> | |
| ELSEIF / ELSEIFNOT <条件> | - 可选 ELSEIF 或 ELSEIFNOT |
| <缩进代码> | |
| 别的 | - 可选的最终 ELSE |

<缩进代码>

案件/案件_快速

CASE 结构按顺序将目标值与可能匹配的列表进行比较。当找到匹配项时,将执行相关代码。

匹配值/范围必须缩进超过 CASE 关键字。多个匹配值/范围可以用逗号分隔符表示。与匹配相关的任何其他代码行
值/范围必须缩进匹配值/范围之后:

案例目标	- 具有目标值的 CASE
<匹配> : <代码>	- 匹配值和代码
<缩进代码>	
<匹配..匹配> : <代码>	- 匹配范围和代码
<缩进代码>	
<匹配>,<匹配..匹配> : <代码>	- 匹配值、范围和代码
<缩进代码>	
其他:<code>	- 可选的其他情况,以防未找到匹配项
<缩进代码>	

CASE_FAST 与 CASE 类似,但它不是按顺序将目标与可能匹配的列表进行比较,而是使用最多 256 个条目的索引跳转表立即分支到适当的代码,节省时间,但可能代价是编译后的代码更大。如果只有连续的匹配值,没有匹配范围,则生成的代码实际上会比正常的具有多个匹配值的 CASE 构造。

为了使 CASE_FAST 编译,匹配值/范围必须是彼此相差 255 以内的唯一常量。

参见上面 “流量控制”下的 CASE_FAST 示例。

重复

所有循环都是通过 REPEAT 构造实现的,它有以下几种形式:

重复	- 永远重复 (如果你不想让齿轮停止并停止驱动其 I/O,则将其放在程序末尾很有用)
<缩进代码>	
重复 <次数>	- 重复 <count> 次,如果 <count> 为零,则跳过 <缩进代码>
<缩进代码>	
重复 <变量> 从 <第一个> 到 <最后一个>	- 重复迭代 <variable> 从 <first> 到 <last>,步长为 +/-1
<缩进代码>	- 完成后,<variable> = <last> +/-1
重复 <变量> 从 <第一个> 到 <最后一个> 步骤 <delta>	- 重复迭代 <variable> 从 <first> 到 <last>,步长为 +/-<delta>
<缩进代码>	- 完成后,<variable> = <last> +/-<delta>
重复直到<条件>	- 当 <条件> 不为零时重复,<条件> 在 <缩进代码> 执行之前进行评估
<缩进代码>	
重复直到<条件>	- 重复直到 <条件> 不为零,<条件> 在 <缩进代码> 执行之前进行评估
<缩进代码>	
重复	- 当 <condition> 不为零时重复,<condition> 在 <indented code> 执行后进行评估
<缩进代码>	
WHILE <条件>	- WHILE 必须与 REPEAT 有相同的缩进
重复	- 重复直到 <条件> 不为零,<条件> 在 <缩进代码> 执行后进行评估
<缩进代码>	
直到<条件>	- UNTIL 必须与 REPEAT 有相同的缩进

在 REPEAT 结构中,有两个特殊指令可用于改变执行过程:NEXT 和 QUIT。NEXT 将立即跳转到 REPEAT 中的点
构造中会做出再次循环的决定,而 QUIT 将退出 REPEAT 构造并继续执行。这些指令通常有条件地使用:

重复	
<缩进代码>	
如果 <条件>	- 可选择强制执行 REPEAT 的下一迭代
下一个	

<缩进代码>

如果 <条件>

辞职

<缩进代码>

- 可选择退出重复

在线 PASM 代码

Spin2 方法可以通过在 PASM 代码前加上 “ORG {\$000..\$12F}”并以 END 终止来执行内联 PASM 代码。

PUB 去() | x

重复

组织机构

GETRND WC

将一个随机位旋转到 x

香港公司

x,#1

结尾

PINWRITE(56 ADDPINS 7, x)

将 x 输出到 P2 评估板的 LED

等待ITMS(100)

您的 PASM 代码将在末尾添加一个 RET 指令,以确保它返回到 Spin2,以防没有早期的 _RET_ 或 RET 执行。

以下是执行内联 PASM 代码的内部 Spin2 过程:

- 保存当前流地址,以便在执行 PASM 代码后进行恢复。
- 将方法的前 16 个长变量 (包括任何参数、返回值和局部变量)从集线器 RAM 复制到 cog 寄存器 \$1E0..\$1EF。
- 将内联 PASM 代码长变量从集线器 RAM 复制到 cog 寄存器,从 ORG 地址开始 (默认为 \$000)。
- 调用 PASM 代码。
- 将 cog 寄存器 \$1E0..\$1EF 中的 16 个 long 恢复回集线器 RAM,以便更新任何已修改的方法变量。
- 恢复流媒体地址并恢复 Spin2 字节码执行。

在您的内联 PASM 代码中,您可以执行以下所有操作:

- 读取和写入以下寄存器区域:
 - \$000..\$12F,您的 PASM 代码将加载到其中。您甚至可以在此范围内的不同地址加载不同的 PASM 程序,并从 Spin2 中调用它们。
 - \$1D8..\$1DF,它们是通用寄存器,名为 PR0..PR7,可用于 PASM 和 Spin2 代码。
 - \$1E0..\$1EF,临时包含该方法的前 16 个长集线器 RAM 变量,并临时分配相同的符号名称。
 - \$1F0..\$1FF,其中包括 IJMP3,IJET3,IJMP2,IJET2,IJMP1,IJET1,PA,PB,PTRA,PTRB,DIRA,DIRB,OUTA,OUTB,INA 和 INB。
 - 避免写入 \$130..\$1D7 和 LUT RAM,因为 Spin2 解释器占用了这些区域。您可以查看 “Spin2_interpreter.spin2”以查看解释器代码。
 - 暂时使用流式传输器。
 - 使用最多 5 级硬件堆栈进行嵌套调用,包括对集线器 RAM 的调用。
 - 声明和引用常规和本地符号。这些符号将无法在 PASM 代码之外访问。
 - 声明 BYTE、WORD 和 LONG 数据。
 - 使用 RES、ORGF 和 FIT 指令。内联 PASM 代码中不允许使用指令 ORG、ORGH、ALIGNW、ALIGNL 和 FILE。
 - 建立一个中断,执行 cog 寄存器 \$000..\$12F 中剩余的代码。Spin2 可容纳中断,并且仅在必要时短暂停止它们。
 - 通过执行 _RET_ 或 RET 指令,随时返回 Spin2。

从 SPIN2 调用 PASM

您可以在 Spin2 中执行CALL(address)以在 cog 寄存器空间或 hub RAM 中执行 PASM 代码。

PUB 去() | x

重复

调用(@随机)

PINWRITE (56 ADDPINS 7,PR0)

等待ITMS(100)

数据保护

奥格

hub PASM 程序将随机位旋转到 pr0

随机 GETRND WC

RET 复制代码

PR0,#1

以下是执行 CALL 的内部 Spin2 过程:

- 保存当前流媒体地址,以便在PASM 代码执行后恢复。
- 调用PASM 代码。
- 恢复流媒体地址并恢复Spin2字节码执行。

在您调用的代码中,您可以执行以下所有操作:

- 读取和写入以下寄存器区域:
 - \$000..\$12F,可能包含您之前加载的 PASM 代码和/或数据。
 - \$1D8..\$1DF,是通用寄存器,名为 PR0..PR7,可用于 PASM 和 Spin2 代码。
 - \$1E0..\$1EF,可用于暂存器,但 Spin2 恢复时可能会被重写。
 - \$1F0..\$1FF,包括 IJMP3,IJET3,IJMP2,IJET2,IJMP1,IJET1,PA,PB,PTRA,PTRB,DIRA,DIRB,OUTA,OUTB,INA 和 INB。
 - 避免写入 \$130..\$1D7 和 LUT RAM,因为 Spin2 解释器占用了这些区域。您可以查看 “Spin2_interpreter.spin2”以查看解释器代码。
 - 暂时使用流媒体。
 - 使用最多 5 级硬件堆栈进行嵌套调用,包括对集线器 RAM 的调用。
 - 建立一个中断,执行 cog 寄存器 \$000..\$12F 中剩余的代码。Spin2 可容纳中断,并且仅在必要时短暂停止中断。
 - 随时通过执行 _RET_ 或 RET 指令返回到 Spin2。

REGLOAD 和 REGEXEC

Spin2 指令REGLOAD(HubAddress)和REGEXEC(HubAddress)用于将 PASM 代码和/或数据块从集线器 RAM 加载到 cog 寄存器中,或者加载并执行。

PASM 代码块和/或数据块前面必须有两个字,这两个字提供起始寄存器和要加载的寄存器数量 (长整型)减 1。

公共去 ()		
重新加载(@chunk)	将自定义块从集线器加载到寄存器中	
重复		
调用(#开始)	在寄存器地址处调用块内的程序	
等待ITMS(100)		
数据保护		
词块	start,finish-start-1	定义块开始和大小-1
组织机构	120美元	org 可以是 \$000..\$130 大小
启动 DRVNRD #56 ADDPINS 7	一些代码	
RET DRVNOT #0 完成	更多代码 + 返回	

REGEXEC 的工作方式与 REGLOAD 类似,但它在加载块之后还会调用块的起始寄存器。

在下面的例子中,REGEXEC 在上部寄存器内存中启动一段代码,该代码设置定时器中断,然后返回到 Spin2。与此同时,随着 Spin2 方法反复每 100ms 随机化引脚 60..63,加载到上部寄存器内存中的代码块使定时器中断永久存在,并每 500ms 切换引脚 56..59,请注意,寄存器 \$000..\$127 仍可供其他代码块使用,并且中断 2 和 3 仍未被使用。

公共去 ()		
REGEXEC(@chunk)		加载自定义块并执行
重复		chunk 启动定时器中断并返回
PINWRITE(60 ADDPINS 3, GETRND())		随机化图钉 60..63
等待ITMS(100)		引脚 56..59 通过中断切换
数据保护		
词块	开始,结束-开始-1 \$128	定义块起始和大小-1
组织机构		org 可以是 \$000..\$130 大小
开始 MOV	IJMP1,#isr	设置 int1 向量
设置INT1 #1		将 int1 设置为 ct-passed-ct1 事件
GETCT PR0 _ret_		获取 ct
ADDCT1 PR0,bigwait		设置初始 ct1 目标,返回 Spin2
DRVNOT #56 附加组件 3		中断服务程序,切换 56..59
ADDCT1 PR0,大等待		设置下一个 ct1 目标
雷蒂1		从中断返回
bigwait 长完成	20_000_000 / 2	RCAST 上 500 毫秒秒

调试

Spin2 编译器包含一个隐秘的调试程序,可以随应用程序自动下载。它使用最后 16KB 的 RAM 以及每个 Spin2 DEBUG 的几个字节语句和每个 PASM DEBUG 语句的一条指令。您可以将 DEBUG 语句放置在应用程序中,其中包含输出命令,这些命令将串行传输应用程序运行时,调试器会检查变量和方程。每次在执行过程中遇到 DEBUG 语句时,都会调用调试器并输出该语句的消息。

在 PNut 中,通过将 Ctrl 键添加到常用的 F10 键“运行”或 F11 键“程序”来启动调试,或者在 Propeller Tool 中通过按 Ctrl+D 启用调试模式,然后照常使用 F10 或 F11。

这将使用所有 DEBUG 语句编译您的应用程序,将调试器添加到下载中,然后打开 DEBUG 输出窗口,该窗口在开始时开始接收消息您的申请。

关于 DEBUG 系统你需要知道的事

- 要使用调试器,必须配置至少 10 MHz 时钟（来自晶振或外部输入）。不能使用 RCFast 或 RCLow。
- 调试器占用了集线器 RAM 的前 16 KB,重新映射到 \$FC000..\$FFFF 并处于写保护状态。位于 \$7C000..\$7FFFF 的集线器 RAM 将不再可用。
- 定义每个 DEBUG 语句的数据都存储在 RAM 顶部 16 KB 的调试器映像中,从而最大限度地减少对应用程序代码的影响。
- 在 Spin2 中,每个 DEBUG 语句都会添加三个字节,加上引用变量和解析 DEBUG 语句中使用的运行时表达式所需的任何代码。
- 在 PASM 中,每个 DEBUG 语句都会添加一条指令（长指令）。
- 当未使用 DEBUG 模式进行编译时,DEBUG 语句会被编译器忽略,因此当未使用调试时无需将其注释掉。
- 如果您的应用程序中不存在 DEBUG 语句,则在启动 cogs 时您仍会收到通知消息。
- 通过通常在 F9..F11 键之前按住 CTRL（在 PNut 中）或使用 CTRL+D（在 Propeller Tool 中）启用调试来调用调试,以编译、下载和编程到闪存。
- 在执行过程中,当遇到 DEBUG 语句时,文本消息以 2 Mbaud 的速率以 8-N-1 格式通过 P62 串行发送。
- DEBUG 消息始终以“CogN”开头,其中 N 是齿轮编号,后跟两个空格,并且始终以 CR+LF（换行符）结尾。
- 您的应用程序中最多可以存在 255 个 DEBUG 语句,因为 BRK 指令用于中断和选择特定的 DEBUG 语句定义。
- 您可以定义多个符号来修改调试器行为:DEBUG_COG.DEBUG_DELAY.DEBUG_BAUD.DEBUG_PIN.DEBUG_TIMESTAMP 等。请参阅表格。
- 每次启动启用调试的 cog 时,都会输出一条调试消息以指示 cog 编号、代码地址 (PTRB)、参数 (PTRA) 和“加载”或“跳转”模式。
- 对于 Spin2,DEBUG 语句可以输出表达式和变量值、集线器字节/字/长数组以及寄存器数组。
- 对于 PASM,DEBUG 语句可以输出寄存器值/数组、集线器字节/字/长数组和常量。使用 PASM 语法:隐含寄存器或 #immediate。
- DEBUG 输出数据可以显示为十进制、十六进制或二进制,有符号或无符号,大小为字节、字、长整型或自动。还支持集线器字符串。
- DEBUG 输出命令显示源和值:“DEBUG(UHEX(x))”可能输出“x = \$123”。
- 输出数据的DEBUG命令可以有多个参数,以逗号分隔:SDEC(x,y,z)和LSTR(ptr1,size1,ptr2,size2)
- 数据之间自动输出逗号:“DEBUG(UHEX_BYTE(d,e,f), SDEC(g))”可能输出“d = \$45, e = \$67, f = \$89, g = -1_024”。
- 所有 DEBUG 输出命令都有备用版本,以“_”结尾,仅输出值:DEBUG(UHEX_BYTE_(d,e,f)) 可能输出“\$45、\$67、\$89”。
- 除了命令之外,DEBUG 语句还可以包含逗号分隔的字符串和字符:DEBUG(We got here! Oh, Nooooo... , 13, 13)
- DEBUG 语句可能包含 IF() 和 IFNOT() 命令,用于控制语句中的进一步输出。初始 IF/IFNOT 将控制整个消息。
- DEBUG 语句可能包含最终的 DLY（毫秒）命令来减慢 cog 的消息传递速度,因为消息的流动速度可能达到每秒 ~10,000 条。
- DEBUG 串行输出可以重定向到不同的引脚,以不同的波特率,以便在其他地方显示/记录。
- LOCK[15] 由调试器分配,并在所有 cog 的调试中断期间使用,以分时共享 DEBUG 串行传输引脚。
- 命令行支持仅 DEBUG 模式:PNut -debug {CommPort if not 1} {BaudRate if not 2_000_000}

DEBUG 语句中使用的命令

条件语句	细节
IF(条件)	如果条件 <> 0,则继续执行 DEBUG 语句中的下一个命令,否则跳过所有剩余命令并输出 CR+LF。如果用作 DEBUG 语句中的第一个命令,IF 将控制该语句的所有输出,包括“CogN +CR+LF。这样,DEBUG 消息就可以被完全抑制,以便您可以过滤重要的内容。
IFNOT(条件)	如果条件 = 0,则继续执行 DEBUG 语句中的下一个命令,否则跳过所有剩余命令并输出 CR+LF。如果用作 DEBUG 语句中的第一个命令,IFNOT 将控制该语句的所有输出,包括“CogN +CR+LF。这样,DEBUG 消息就可以被完全抑制,以便您可以过滤重要的内容。

字符串输出 *	细节	输出
ZSTR(中心指针)	在 hub_pointer 处输出以零结尾的字符串	“你好! ”
LSTR(中心指针,大小)	在 hub_pointer 处输出 ‘size’ 个字符的字符串	“再见。”

十进制输出,无符号 *	细节	最小输出	最大输出
UDEC(值)	输出无符号十进制值	0	4_294_967_295
UDEC_BYTE(值)	输出字节大小的无符号十进制值	0	255
UDEC_WORD(值)	输出字长无符号十进制值	0	65_535
UDEC_LONG(值)	输出长整型无符号十进制值	0	4_294_967_295
UDEC_REG_ARRAY(reg_pointer,大小)	将寄存器数组输出为无符号十进制值	0	4_294_967_295
UDEC_BYTE_ARRAY(hub_pointer,size) 将 hub 字节数组输出为无符号十进制值		0	255
UDEC_WORD_ARRAY(hub_pointer,size) 将集线器数组输出为无符号十进制值		0	65_535
UDEC_LONG_ARRAY(hub_pointer,size) 将 hub 长数组输出为无符号十进制值		0	4_294_967_295
十进制输出,有符号 *	细节	最小输出	最大输出

SDEC(值)	输出有符号十进制值	-2_147_483_648	2_147_483_647
SDEC_BYTE(值)	输出字节大小的有符号十进制值	-128	127
SDEC_WORD(值)	输出字长有符号十进制值	-32_768	32_767
SDEC_LONG(值)	输出长整数有符号十进制值	-2_147_483_648	2_147_483_647
SDEC_REG_ARRAY(reg_pointer,大小)	将寄存器数组输出为有符号的十进制值	-2_147_483_648	2_147_483_647
SDEC_BYTE_ARRAY(hub_pointer,size) 输出 hub 字节数组作为有符号的十进制值		-128	127
SDEC_WORD_ARRAY(hub_pointer,size) 以有符号十进制值形式输出 hub 字数组		-32_768	32_767
SDEC_LONG_ARRAY(hub_pointer,size) 输出 hub 长数组作为有符号十进制值		-2_147_483_648	2_147_483_647
十六进制输出,无符号*	细节	最小输出	最大输出
UHEX(值)	输出自动调整大小的无符号十六进制值	\$0	\$FFFF_FFFF
UHEX_BYTE(值)	输出字节大小的无符号十六进制值	\$00	\$FF
UHEX_WORD(值)	输出字大小的无符号十六进制值	\$0000	\$FFFF
UHEX_LONG(值)	输出长尺寸无符号十六进制值	\$0000_0000	\$FFFF_FFFF
UHEX_REG_ARRAY(reg_pointer,大小)	将寄存器数组输出为无符号十六进制值	\$0000_0000	\$FFFF_FFFF
UHEX_BYTE_ARRAY(hub_pointer,size) 将 hub 字节数组输出为无符号十六进制值		\$00	\$FF
UHEX_WORD_ARRAY(hub_pointer,size) 将集线器字数组输出为无符号十六进制值		\$0000	\$FFFF
UHEX_LONG_ARRAY(hub_pointer,size) 输出 hub 长数组作为无符号十六进制值		\$0000_0000	\$FFFF_FFFF
十六进制输出,有符号*	细节	最小输出	最大输出
SHEX(值)	输出自动调整大小的有符号十六进制值	-\$8000_0000	\$7FFF_FFFF
SHEX_BYTE(值)	输出字节大小的有符号十六进制值	-\$80	\$7F
SHEX_WORD(值)	输出字大小的有符号十六进制值	-8000美元	\$7FFF
SHEX_LONG(值)	输出长尺寸有符号十六进制值	-\$8000_0000	\$7FFF_FFFF
SHEX_REG_ARRAY(reg_pointer,大小)	将寄存器数组输出为有符号的十六进制值	-\$8000_0000	\$7FFF_FFFF
SHEX_BYTE_ARRAY(hub_pointer,size) 将 hub 字节数组输出为有符号的十六进制值		-\$80	\$7F
SHEX_WORD_ARRAY(hub_pointer,size) 输出集线器字数组作为有符号的十六进制值		-8000美元	\$7FFF
SHEX_LONG_ARRAY(hub_pointer,size) 输出集线器长数组作为有符号的十六进制值		-\$8000_0000	\$7FFF_FFFF
二进制输出,无符号*	细节	最小输出	最大输出
UBIN(值)	输出自动调整大小的无符号二进制值	%0	%11111111_11111111_11111111_11111111
UBIN_BYTE(值)	输出字节大小的无符号二进制值	%00000000	%11111111
UBIN_WORD(值)	输出字大小的无符号二进制值	%00000000_00000000	%11111111_11111111
UBIN_LONG(值)	输出长尺寸无符号二进制值	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
UBIN_REG_ARRAY(reg_pointer,大小)	将寄存器数组输出为无符号二进制值	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
UBIN_BYTE_ARRAY(hub_pointer,size) 将 hub 字节数组输出为无符号二进制值		%00000000	%11111111
UBIN_WORD_ARRAY(hub_pointer,size) 将集线器字数组输出为无符号二进制值		%00000000_00000000	%11111111_11111111
UBIN_LONG_ARRAY(hub_pointer,size) 输出 hub 长数组作为无符号二进制值		%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
二进制输出,有符号*	细节	最小输出	最大输出
SBIN(值)	输出自动调整大小的有符号二进制值	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_BYTE(值)	输出字节大小的有符号二进制值	-%10000000	%01111111
SBIN_WORD(值)	输出字大小的有符号二进制值	-%10000000_00000000	%01111111_11111111
SBIN_LONG(值)	输出长型有符号二进制值	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_REG_ARRAY(reg_pointer,大小)	将寄存器数组输出为有符号的二进制值	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_BYTE_ARRAY(hub_pointer,size) 将 hub 字节数组输出为有符号二进制值		-%10000000	%01111111
SBIN_WORD_ARRAY(hub_pointer,size) 将集线器字数组输出为有符号二进制值		-%10000000_00000000	%01111111_11111111
SBIN_LONG_ARRAY(hub_pointer,size) 输出 hub 长数组作为有符号二进制值		-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111

延迟调速信息	细节
DLY(毫秒)	延迟几毫秒以减慢此齿轮的连续消息输出。DLY 仅允许作为 DEBUG 语句,因为它在延迟之前释放了 LOCK[15],从而允许其他 cog 捕获 LOCK[15],以便它们可以控制 DEBUG 串行传输引脚并输出自己的 DEBUG 消息。

* 这些命令接受多个参数或多组参数。具有相同名称但以 “_” 结尾的替代命令也可用于仅输出值（即 ZSTR_，

请参阅 UDEC_ 以了解如何使用 UDEC 函数。

可以定义修改 DEBUG 行为的符号

CON 符号	默认	目的
下载波特率	2_000_000	设置下载波特率。
调试_COGS	%11111111	选择哪些 cogs 启用了调试中断。位 7..0 启用 cogs 7..0 中的调试中断。
调试延迟	0	在应用程序运行并开始传输 DEBUG 消息之前设置一个以毫秒为单位的延迟。
调试引脚	62	设置 DEBUG 串行输出引脚。要打开 DEBUG 窗口,DEBUG_PIN 必须为 62。
调试波特率	DOWNLOAD_BAUD 设置	DEBUG 波特率。如果 DEBUG_BAUD 小于 DOWNLOAD_BAUD,可能需要添加 DEBUG_DELAY。
调试时间戳	不明确的	通过声明此符号,每个 DEBUG 消息将带有 64 位 CT 值的时间戳。
调试日志大小	0	设置收集 DEBUG 消息的“DEBUG.log”文件的最大大小。值为 0 将禁止生成日志文件。
调试_左	(动态的)	设置 DEBUG 消息窗口出现的左屏幕坐标。
调试顶部	(动态的)	设置 DEBUG 消息窗口出现的顶部屏幕坐标。
调试_宽度	(动态的)	设置 DEBUG 消息窗口的宽度。
调试高度	(动态的)	设置 DEBUG 消息窗口的高度。
调试_显示_左	0	设置任何 DEBUG 显示将出现的整体左屏幕偏移 (添加到每个 DEBUG 显示中的“POS”x 坐标)。
调试显示顶部	0	设置任何 DEBUG 显示将出现的整体顶部屏幕偏移 (添加到每个 DEBUG 显示中的“POS”y 坐标)。
调试_WINDOWS_OFF	0	如果设置为非零值,则禁止在下载后打开任何 DEBUG 窗口。

Spin2 中的简单 DEBUG 示例

CON _clkfreq = 10_000_000	设置 10 MHz 时钟 (假设 20 MHz 晶振)
PUB 去() 我	
重复 i 从 0 到 9	从 0 数到 9
调试 (UDEC(i))	调试,输出 i

使用 Ctrl-F10 运行时,将打开 “调试”窗口并显示以下内容：

Cog0 INIT \$0000_0000 \$0000_0000 加载
Cog0 INIT \$0000_0D6C \$0000_10BC 跳转
脉0 i = 0
Cog0 i = 1
考格0 i = 2
知识0 i = 3
考格0 i = 4
齿形0 i=5
齿形0 i=6
认知0 i = 7
考格0 i = 8
认知0 i = 9

在报告的第一行中,您会看到 Cog0 从 \$00000 加载 Spin2 设置代码。在第二行中,Spin2 解释器从 \$00D58 启动,其堆栈空间从 \$0101C 开始。
之后,Spin2 程序运行,您会看到 “i”从 0 迭代到 9。

如果将 REPEAT 中的 “9”改为 “99” ,数据将滚动得太快而无法读取,但通过在 DEBUG 语句末尾添加 DLY 命令,可以减慢输出速度：

调试 (udec (i) ,dly (250))	调试,每次报告后延迟 250ms 输出 i
---------------------------	-----------------------

假设您想限制输出的消息,以便只显示 “i”的奇数值。您可以在 DEBUG 语句的开头使用 IF 来检查 “i”的最低有效位。

当 IF 为假时,不会输出任何消息,只显示 i 的奇数值：

调试 (如果 (i&1) ,udec (i) ,dly (250))	调试,每次报告后仅输出奇数 i 值,延迟 250ms
-------------------------------------	----------------------------

PASM 中的简单 DEBUG 示例

CON _clkfreq = 10_000_000	设置 10 MHz 时钟 (假设 20 MHz 晶振)
---------------------------	-----------------------------

调试和中断

DEBUG 语句期间收到的中断请求将在 DEBUG 完成后执行,但响应时间可能会出现偏差,导致中断的重新触发设置不会发生正确。高频周期性智能引脚中断尤其容易出现此问题。想象一下,您在正常 ISR (中断服务例程)中执行 AKPIN 指令以丢弃 INA/INB 信号,以便智能引脚可以使其再次升高,从而触发新的中断。同时,在 AKPIN 之后和 RETIx 之前,智能引脚触发,将 INA/INB 升高。这是仅当您的循环帧定时偏离 DEBUG 语句时才会发生这种情况。由于此中断发生在 ISR 繁忙时,因此不会出现。这将导致中断停止循环。但是 CT 中断不容易出现此问题,因为它们有 \$8000_0000 个时钟周期需要识别。为了解决智能引脚重新触发问题,您可以在 INA/INB-high 上触发,而不是 INA/INB-rise,但这可能会导致智能引脚配置出现性能问题。

解决此 DEBUG/中断困境的一种安全方法是仅从未在后台执行 ISR 的 cog 执行 DEBUG 语句。如果 ISR 可以容忍时序偏差并且没有挂起中断循环的风险,您可以使用一些已了解的中断时序退化来执行 DEBUG 语句。

图形调试显示

DEBUG 消息可以调用内置于工具中的特殊图形 DEBUG 显示。这些图形显示各自采用独特的窗口形式。实例化后,显示可以连续输入数据以生成动画可视化。这些显示对于开发和调试非常方便,因为可以在其原生上下文中查看各种数据类型。最多 32 个图形显示可同时运行。

当 DEBUG 消息包含反引号 (`) 字符 (ASCII \$60) 时,会将包含从反引号到消息结尾的所有内容的字符串发送到图形 DEBUG 显示屏解析器。解析器会查找几种不同的元素类型,并将任何逗号视为空格:

元素类型	例子	描述
显示类型	逻辑、范围、情节、位图	这是您希望实例化的图形 DEBUG 显示类型的正式名称。
未知符号	韓識显示	必须为每个图形 DEBUG 显示实例赋予一个唯一的符号名称。
实例名称	韓識显示	一旦实例化,图形 DEBUG 显示实例将通过其符号名称引用。
关键词	标题、POS、尺寸、样品	关键字用于配置显示。关键字后面可能跟有数字、字符串和其他关键字。
数字	1024, \$FF,%1010	数字可以用十进制、十六进制 (\$)和二进制 (%)来表示。
细绳	“这是一个字符串”	字符串在单引号内表示。

在了解所有这些如何组合在一起之前,我们需要了解一些可用于显示的特殊 DEBUG-display 语法。当 DEBUG 中的第一个字符语句是反引号。这会导致 DEBUG 语句中的所有内容都被视为字符串,除非后续反引号充当“转义”字符以允许正常或简写 DEBUG 命令。

DEBUG 语句 (v=100,BYTE[a]=1,2,3,4,5)	DEBUG 消息输出	笔记
DEBUG(` LOGIC MyDisplay SAMPLES ,SDEC_(v)) Cog0 ` LOGIC MyDisplay SAMPLES 100		常规 DEBUG 语法可以驱动 DEBUG 显示,但它不是最佳的。
调试 (` LOGIC MyDisplay SAMPLES 100)	` LOGIC MyDisplay 样本 100	DEBUG-display 语法更简单,并且在输出中省略了“CogN”。
DEBUG (` LOGIC MyDisplay SAMPLES ` (v))	` LOGIC MyDisplay 样本 100	十进制数使用 ` (值)表示法输出。SDEC_ 的缩写。
DEBUG (` LOGIC MyDisplay SAMPLES` \$ (v))	` LOGIC MyDisplay 样本 64 美元	十六进制数使用 `(value) 符号输出。UHEX_ 的缩写。
DEBUG (` LOGIC MyDisplay SAMPLES` % (v))	` LOGIC MyDisplay 样本 %1100100	二进制数使用 `(value) 符号输出。UBIN_ 的缩写。
DEBUG (` LOGIC MyDisplay TITLE ` #(v))	` LOGIC MyDisplay TITLE d	字符使用 `(value) 符号输出。
DEBUG (` MyDisplay` UDEC_BYTE_ARRAY_ (@a,5))	`我的显示器 1, 2, 3, 4, 5	常规 DEBUG 命令也可以跟在反引号后面。

使用图形 DEBUG 显示有两个步骤。首先,必须实例化它们;其次,必须输入它们:

使用显示器:	第一	第二	第三	第四	笔记
首先,实例化它。	`	显示类型	未知符号	关键字,数字,字符串未知符号变为实例名称。	
然后,喂它。	`	实例名称	关键字、数字、字符串		多个显示器可以输入相同的数据。

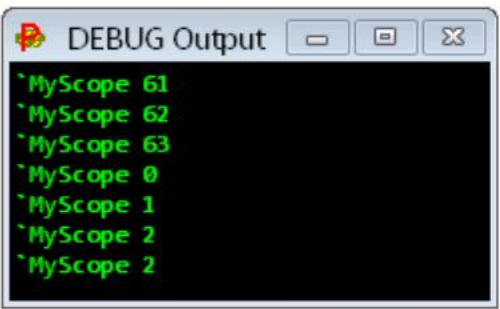
为了将所有这些结合起来,让我们在 SCOPE 显示屏上显示锯齿波:


```
CON _clkfreq = 10_000_000

PUB 去()我

    调试 ( `范围 MyScope 大小 254 84 样本 128)
    调试 ( `MyScope  Sawtooth  0 63 64 10%1111)

    重复
        调试 ( `MyScope` (i&63) )
        我++
        等待毫秒(50)
```



在上面的例子中,实例化了一个名为 MyScope 的 SCOPE,其尺寸为 254 x 84 像素,显示 128 个样本。之所以选择宽度为 254,是因为样本编号为 0..127,我希望

将它们以恒定的两像素间距 (127 * 2 = 254) 隔开。选择 84 的高度,这样波形的上方和下方将有 10 个像素,波形的高度为 64 像素。

定义了一个名为 “锯齿”的通道,为了便于显示,其底部值为 0,顶部值为 63,在此范围内高度为 64 像素,并且比底部高出 10 像素

范围窗口。%1111 启用顶部和底部图例值以及顶部和底部线条。在 REPEAT 块中,SCOPE 被输入 0..63 的重复模式,这构成了

锯齿波。示波器每次收到一个值时都会更新其显示。如果定义了八个通道,而不是一个,它会在每八个值时更新显示

已接收,绘制所有八个通道。

目前,已实现了以下图形 DEBUG 显示,但将来会添加更多内容:

显示类型	描述
逻辑	具有单位和多位标签的逻辑分析仪,1..32 个通道,可触发模式
范围	具有 1..8 个通道的示波器,可以触发滞后电平
范围_XY	XY 示波器具有 1..8 个通道、0..512 个样本的持久性、极坐标模式、对数刻度模式
快速傅里叶变换	具有 1..8 通道、4..2048 个点、窗口结果、对数刻度模式的快速傅里叶变换
斯派克	具有 4..2048 点 FFT、窗口结果、相位着色和对数刻度模式的光谱仪
阴谋	具有笛卡尔和极坐标模式的通用绘图仪
学期	文本终端最多可容纳 300 x 200 个字符、6..200 点字体大小、4 种同时配色方案
位图	位图,1..2048 x 1..2048 像素,1/2/4/8/16/32 位像素,具有 19 种色彩系统、15 种方向/自动滚动模式、独立的 X 和 Y 像素大小为 1..256
MIDI	带有 1..128 个键、力度显示、可变屏幕刻度的钢琴键盘

以下是每个 DEBUG 显示类型的详细说明。

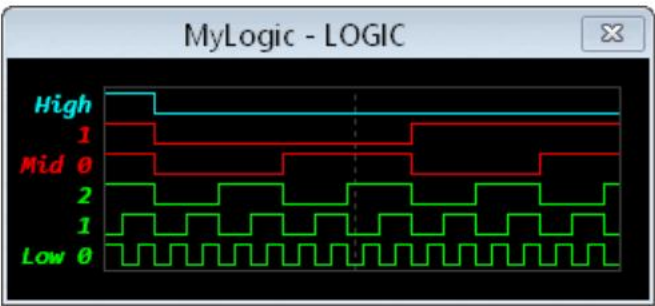
逻辑显示具有单位和多位标签的逻辑分析仪,1..32 个通道,可触发模式

```
CON _clkfreq = 10_000_000

PUB 去()我

    调试 ( ` LOGIC MyLogic SAMPLES 32 低 3 中 2 高 )
    调试 ( ` MyLogic TRIGGER $07 $04 HOLDOFF 2)

    重复
        调试 ( ` MyLogic` (i&63) )
        我++
        等待 (25)
```



逻辑实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
样本 4_to_2048	设置要跟踪和显示的样本数量。	三十二
间距 2_to_32	设置样本间距。显示的宽度为 SAMPLES * SPACING。	8
比率 1_to_2048	设置每次显示更新之前的样本数（或触发器数,如果启用）。	1
线尺寸 1_到_7	设置线条大小。	1
文本大小 6_至_200	设置图例文字大小。文字高度决定逻辑层的高低。	编辑器文字大小
COLOR 背景颜色 {网格颜色}	设置背景和网格颜色*。	黑色,灰色 4
‘名称’ {1_to_32 {颜色}}	设置第一个/下一个通道或组名、可选位数、可选颜色*。	1、默认颜色
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<无>
逻辑喂养	描述	默认
触发掩码匹配sample_offset	触发开启 (数据和掩码) = 匹配。如果掩码 = 0,则禁用触发器。	0.1.样本 / 2
延迟 2_to_2048	设置从触发到触发所需的最小样本数。	样品
数据	数值数据以 LSB 优先的方式应用到通道。	
清除	清除样本缓冲区和显示屏,等待新数据。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

* 颜色是 rgb24 值,否则为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,后跟可选的 0..15 亮度（默认为 8）。

LOGIC 显示器可用于显示高速捕获的数据。在下面的示例中,P2 使用智能引脚以 333 Mbaud 生成 8-N-1 串行。此比特流可以流式处理器捕获的数据。在每个时钟上,流式处理器将记录智能引脚的 IN 信号及其输出状态,从相邻引脚读取。每次它获得四个 2 位样本集时,它都会 RFBYTE 将它们保存到集线器 RAM ,形成连续的字节、字和长整型。通过调用 LONGS_2BIT 打包数据模式,我们可以让 LOGIC 显示器解压两位从长材中抽取样品集,每根长材产生 16 组。

```
CON _clkfreq = 333_333_333  速度非常快,3ns 时钟周期
    rxpin      = 24   偶数针
    txpin      = rxpin+1  奇数引脚
    samps      = 32   16 个样本的倍数
    bufflongs = samps / 16  每个 long 保存 16 个 2 位样本
    模式      = $D0800000 + rxpin << 17 + samps  流光模式

VAR buff[bufflongs]

PUB go() | i,buffaddr

    调试 ( `逻辑串行样本` (samps)间距 12  TX   IN  longs_2bit)
    调试 ( `串行触发 %10 %10 22)
    缓冲区地址 := @buff

    重复
        组织
        沃平          ##+1<<28,#rxpin          rxpin 在 rxpin+1 输入 txpin

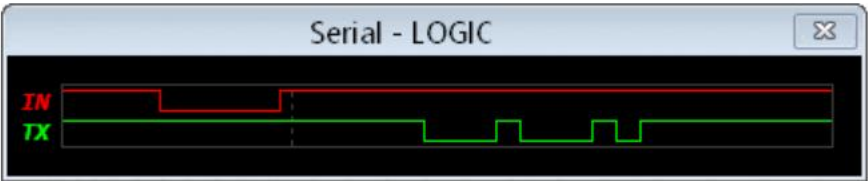
        响鸣 响      #%01_11110_0,#txpin  为 txpin 设置异步 tx 模式
        响  dirh     #1<<16+8-1,#txpin  设置波特率=sysclock/1 和 size=8
                        #txpin  启用智能密码

        写快速初      #0,buffaddr          将写入速度设置为 buff
        始化          ##xmode,#0          开始捕获 2 位样本

        维平          我,# txpin          传输串行字节

        等待
        结尾          等待流光捕获完成

    调试 ( `串行` uhex_long_array_ (@buff,bufflongs) )
    我++
    等待 (20)
```



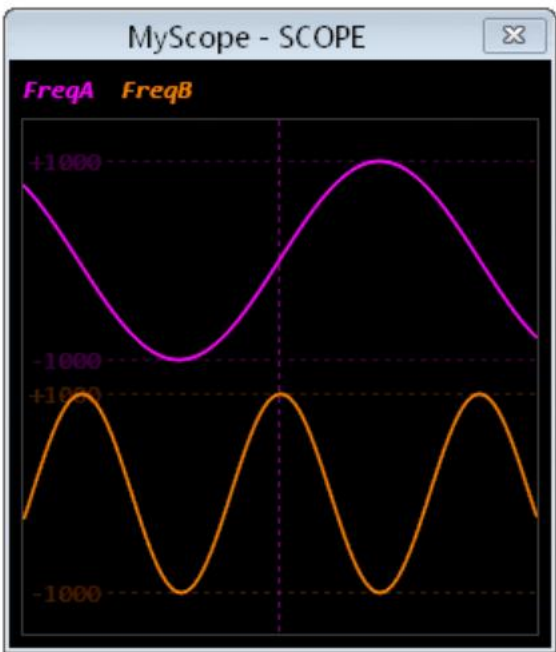
示波器显示示波器具有 1..8 个通道,可以触发滞后电平

```
CON _clkfreq = 100_000_000

PUB go() | a,af,b,bf

    调试 (`SCOPE MyScope)
    调试 (`MyScope  FreqA  -1000 1000 100 136 15 洋红色)
    调试 (`MyScope  FreqB  -1000 1000 100 20 15 ORANGE)
    调试 (`MyScope TRIGGER 0 HOLDOFF 2)

    重复
        a:= qsin (1000,af++,200)
        b:= qsin(1000, bf++, 99)
        调试 (`MyScope` (a,b) )
        韦特斯(200)
```



SCOPE 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
尺寸 宽度 高度	设置显示尺寸（32..2048 x 32..2048）	255,256
样本 16_to_2048	设置要跟踪和显示的样本数量。	256
比率 1_to_2048	设置每次显示更新之前的样本数（或触发器数,如果启用）。	1
点大小 0_至_32	以像素为单位设置点的大小来显示精确的样本点。	0
线大小 0_到_32	设置连接采样点的线的大小（以半像素为单位）。	3
文本大小 6_至_200	设置图例文字大小。	编辑器文字大小
COLOR 背景颜色 {网格颜色}	设置背景和网格颜色*。	黑色,灰色 4
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<无>
喂养范围	描述	默认
name {min {max {y_size {y_base {legend {color}}}}}设置第一个/下一个通道名称、最小值、最大值、y 尺寸、y 基准、图例和颜色*。	图例为 %abcd,其中 %a 到 %d 分别表示最大图例、最小图例、最大行数、最小行数。	完整,无图例，默认颜色
触发通道 {arm_level {trigger_level {offset}}}	设置触发通道、臂电平、触发电平和右偏移。如果通道=-1,则禁用。	-1, -1,0,宽度/2
延迟 2_to_2048	设置从触发到触发所需的最小样本数。	样品
数据	数值数据按升序应用于通道。	
清除	清除样本缓冲区和显示屏,等待新数据。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

* 颜色是 rgb24 值,否则为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,后跟可选的 0..15 亮度（默认为 8）。

SCOPE_XY 显示

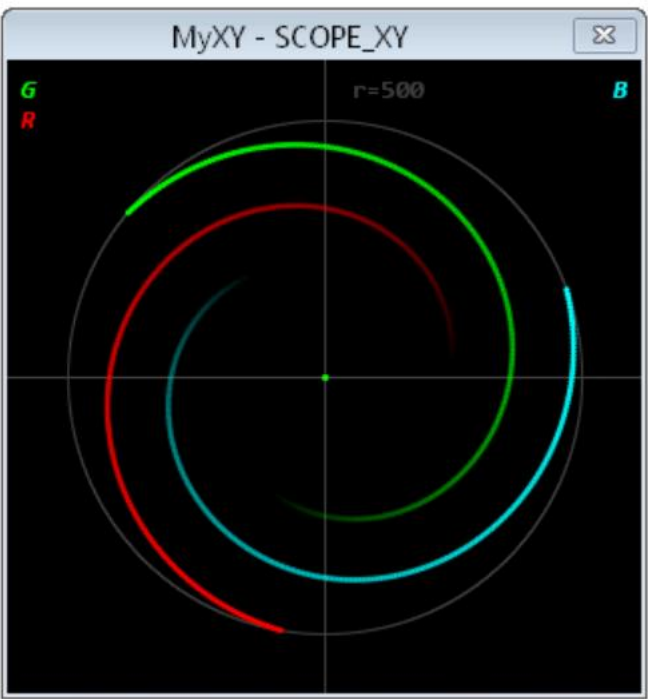
XY 示波器具有 1..8 个通道、1..512 个样本的持久性、极坐标模式、对数刻度模式

```
CON _clkfreq = 100_000_000

PUB 去()我

  调试 ( ` SCOPE_XY MyXY RANGE 500 POLAR 360  G    R    B  )

  重复
    重复 i 从 0 到 500
      调试 ( ` MyXY` (i,j,i,i+120,i,j,i+240) )
      等待消息(5)
```



SCOPE_XY 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
尺寸半径	以像素为单位设置显示半径。	128
范围 1 至 7FFFFFFF	设置传入数据的单位圆半径	\$7FFFFFFF
样本 0_to_512	设置要跟踪并显示持久性的样本数量。使用 0 表示无限持久性。	256
比率 1_to_512	设置每次显示更新之前的样本数。	1
点尺寸 2_至_20	以半像素为单位设置点的大小来显示样本点。	6
文本大小 6_至_200	设置图例文字大小。	编辑器文字大小
COLOR 背景颜色 {网格颜色}	设置背景和网格颜色*。	黑色,灰色 4
极地 {twopi {偏移}}	设置极坐标模式,twopi 值和偏移量。对于 twopi 值为 \$100000000 或 -\$100000000,请使用 0 或 -1。	1亿美元,0
逻辑量表	设置对数尺度模式来放大单位圆内的点。	<关闭>
‘名称’ {颜色}	设置第一个/下一个通道名称并可选地为其分配颜色*。	默认颜色
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<无>
SCOPE_XY 喂食	描述	默认
坐标	XY 数据对按升序应用到通道。在极坐标模式下,x=长度,y=角度。	
清除	清除样本缓冲区和显示屏,等待新数据。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

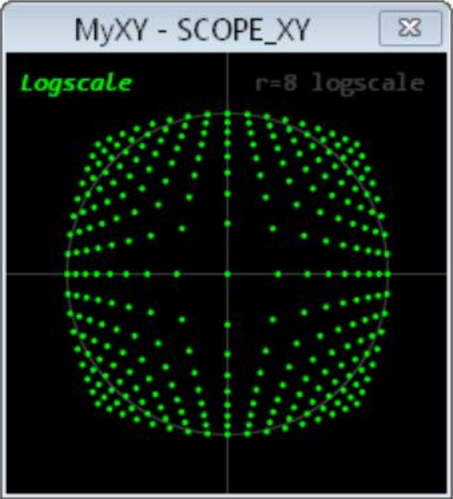
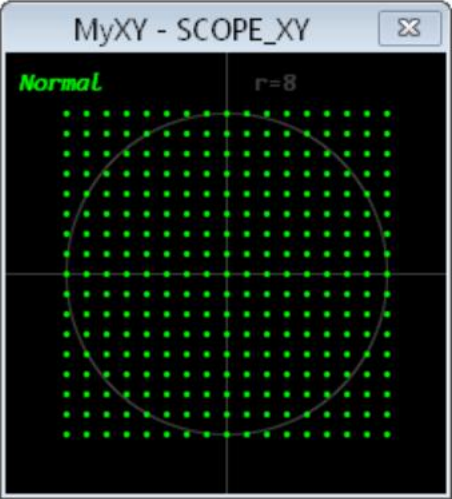
* 颜色是 rgb24 值,否则为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,后跟可选的 0..15 亮度（默认为 8）。

```
CON _clkfreq = 10_000_000          正常模式

PUB go() | x,y
  调试 ( ` SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0  正常  )
  重复 x 从 -8 到 8
    重复 y 从 -8 到 8
      调试 ( ` MyXY` (x,y) )
```

```
CON _clkfreq = 10_000_000          LOGSCALE 模式放大低级细节

PUB go() | x,y
  调试 ( ` SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0 LOGSCALE  对数尺度  )
  重复 x 从 -8 到 8
    重复 y 从 -8 到 8
      调试 ( ` MyXY` (x,y) )
```



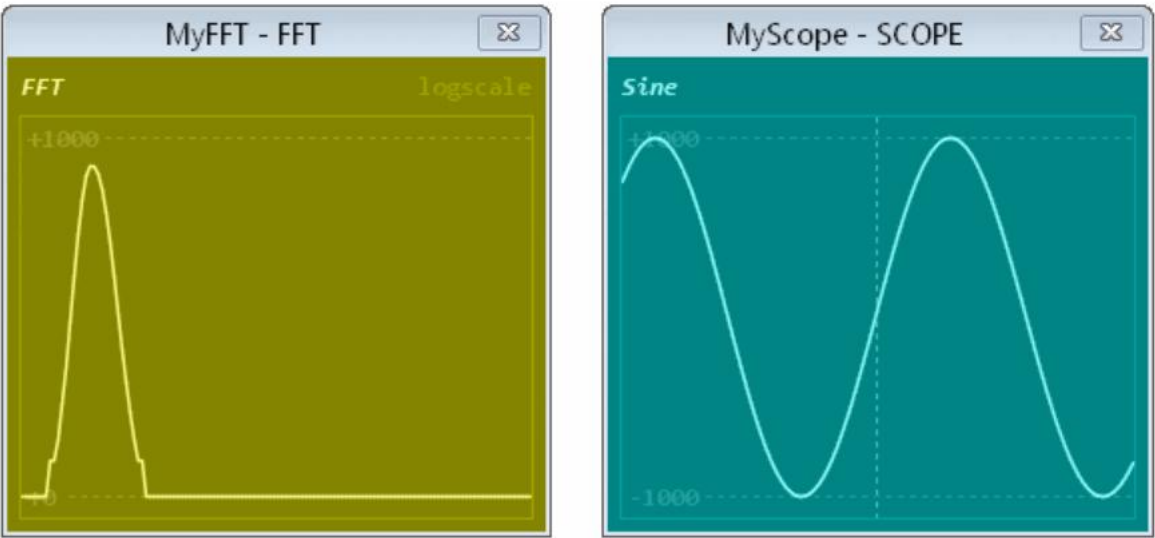
FFT显示具有 1..8 通道、4..2048 个点、窗口结果、对数刻度模式的快速傅里叶变换

```
CON _clkfreq = 100_000_000

PUB go() | i,j,k
{
    设置 FFT
    调试 ( ` FFT MyFFT 大小 250 200 样本 2048 0 127 速率 256 对数尺度颜色黄色 4 黄色 5)
    调试 ( ` MyFFT  FFT  0 1000 180 10 15 黄色 12)
}

{
    设置范围
    调试 ( ` scope MyScope POS 300 0 大小 255 200 颜色青色 4 青色 5)
    调试 ( ` MyScope  正弦  -1000 1000 180 10 15 CYAN 12)
    调试 ( ` MyScope TRIGGER 0)
}

重复
{
    j += 1550 + qsin(1300,i++,31_000)
    k:= qsin (1000,j,50_000)
    调试 ( ` MyFFT MyScope ` (k) )
    韦特斯(100)
}
```



FFT 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
尺寸 宽度 高度	设置显示尺寸（32..2048 x 32..2048）	256,256
SAMPLES 4_to_2048 {first {last}}	设置 2 个 FFT 输入点,加上要显示的第一个和最后一个结果值。	512, 0, 255
比率 1_to_2048	设置每次显示更新之前的样本数。	样品
点大小 0_至_32	以像素为单位设置点的大小来显示精确的样本点。	0
LINESIZE neg32_to_32	设置连接采样点的线的大小（以半像素为单位）。负的线大小将产生孤立的垂直线。	3
文本大小 6_至_200	设置图例文字大小。	编辑器文字大小
COLOR 背景颜色 {网格颜色}	设置背景和网格颜色*。	黑色,灰色 4
逻辑量表	设置对数尺度模式来放大低级结果。	<关闭>
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<无>
FFT 进料	描述	默认
名称 {mag {max {y_size {y_base {图例 {颜色}}}}}	设置第一个/下一个通道名称、放大倍数（2 ,n = 0..11）、最大振幅、y 尺寸、y 基底、图例和颜色*。图例为 %abcd,其中 %a 到 %d 启用最大图例、最小图例、最大线、最小线。	完整,无图例,默认颜色
数据	数值数据被输入到通道的滑动汉宁窗口中,FFT 根据该窗口计算功率水平。	
清除	清除样本缓冲区和显示屏,等待新数据。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

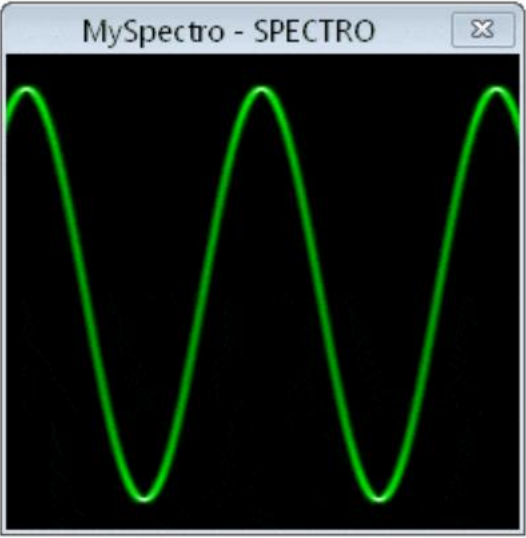
* 颜色是 rgb24 值,否则为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,后跟可选的 0..15 亮度（默认为 8）。

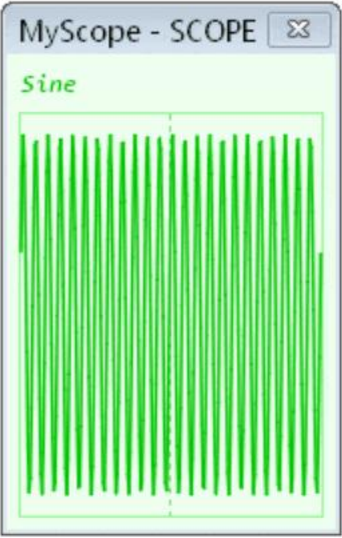
光谱显示具有 4..2048 点 FFT、相位着色和对数刻度模式的光谱仪

```
CON _clkfreq = 100_000_000

PUB go() | i,j,k
    '
    ' 设置 SPECTRO
    调试 (`SPECTRO MySpectro SAMPLES 2048 0 236 RANGE 1000 LUMA8X GREEN)
    '
    ' 设置范围
    调试 (`范围 MyScope POS 280 大小 150 200 颜色绿色 15 绿色 12)
    调试 (`MyScope 正弦 -1000 1000 180 10 0 绿色 6)
    调试 (`MyScope TRIGGER 0)

    重复
        j += 2850 + qsin(2500,i++,30_000)
        k:= qsin (1000,j,50_000)
        调试 (`MySpectro MyScope` (k) )
        韦特斯(100)
```

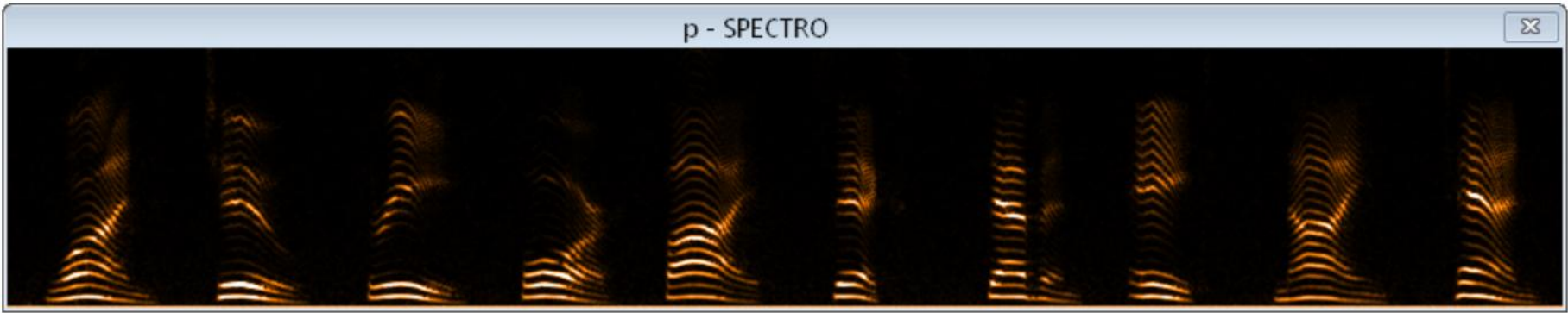




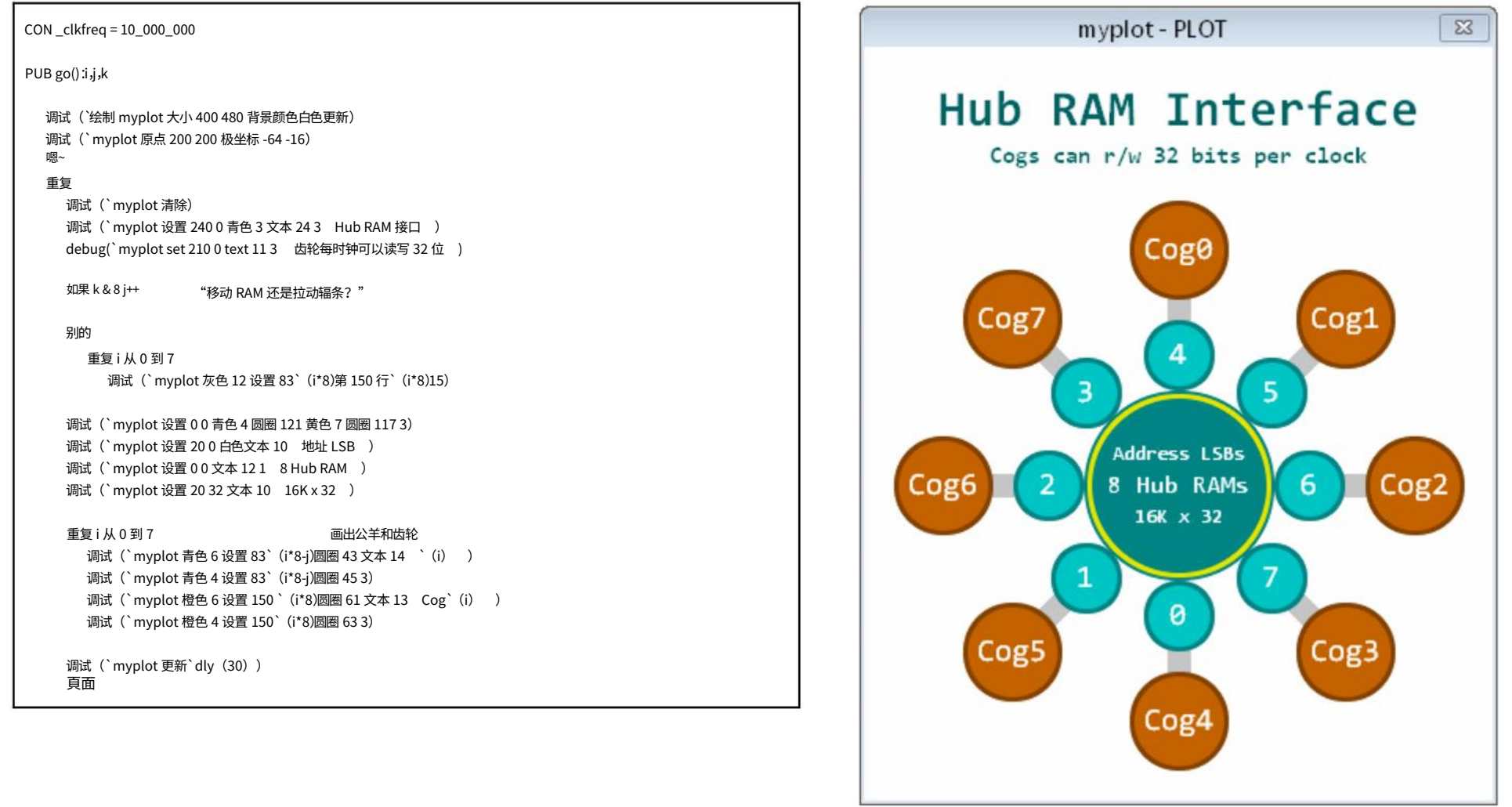
SPECTRO 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
样本 4_to_2048 {first {last}}	设置 2 个 FFT 输入点,加上要显示的第一个和最后一个结果值（定义显示高度）。	512, 0, 255
深度 1_到_2048	设置要显示的垂直线 FFT 结果的数量（定义显示宽度）。	256
MAG 0_至_11	设置放大倍数（2 ⁿ ,n = 0..11）。	0
范围饱和度和功率	设置像素亮度饱和的功率级别。	\$7FFFFFFF
比率 1_to_2048	设置每次显示更新之前的样本数。	样品 / 8
跟踪 0_到_15	设置跟踪模式（参见位图显示中的跟踪动画）。	15 (向右、向上、滚动)
DOTSIZE width_and_height {height}	将光谱仪像素宽度和像素高度一起设置,或单独设置它们。	1,1
luma_or_hsv {颜色或相位}	将配色方案设置为带有颜色*的 LUMA8(W)(X),或带有 0..255 相位着色偏移的 HSV16(W)(X)。	LUMA8X 橙色
逻辑量表	设置对数尺度模式来放大低级结果。	<关闭>
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<无>
斯派克喂料	描述	默认
数据	数值数据被输入到滑动汉宁窗中,FFT 根据该窗口计算功率和相位。	
清除	清除样本缓冲区和显示屏,等待新数据。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

* 颜色为橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色。

下面,SPECTRO 显示器从连接到麦克风的针脚输入 ADC 样本。这是从“1”到“10”的口头计数的光谱图。“1”在左边,“10”在右侧。水平趋势线之间的垂直距离是声门音高。较大的亮度趋势是声音共振峰。这让我们了解了我们的耳朵如何感知语音：



地块显示具有笛卡尔和极坐标模式的通用绘图仪



PLOT 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
尺寸 宽度 高度	设置显示宽度（32..2048）和高度（32..2048）。	256,256
DOTSIZE 宽度和高度 {高度}	一起设置显示像素宽度和像素高度,或者单独设置它们。	1,1
lut1_to_rgb24	设置色彩模式。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式,使用 rgb24 颜色加载 LUT,使用 HEX_LONG_ARRAY_ 加载颜色。	默认颜色 0..7
背景颜色	根据当前颜色模式设置背景颜色。 *	黑色的
更新	设置更新模式。只有输入“更新”命令时,显示才会更新。	自动更新
小区饲喂	描述	默认
lut1_to_rgb24	设置色彩模式。	RGB24 RGB 图像
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式,使用 rgb24 颜色加载 LUT,使用 HEX_LONG_ARRAY_ 加载值。	默认颜色 0..7
背景颜色	根据当前颜色模式设置背景颜色。 *	黑色的
颜色 颜色	根据当前颜色模式设置绘图颜色。在 TEXT 之前使用可更改文本颜色。 *	青色
黑色/白色或橙色/蓝色/绿色/青色/ 红色/洋红色/黄色/灰色 {亮度}	设置绘图颜色和可选的 0..15 亮度（橙色..灰色）（默认为 8）。	青色
不透明度	设置点、线、圆、椭圆、方框和 OBOX 绘图的不透明度级别。0..255 = 清晰..不透明。	255
精确的	切换精确模式,其中 DOT 和 LINE 的线条大小和 (x,y) 以 256 个像素表示。	已禁用
LINESIZE 尺寸	设置 DOT 和 LINE 绘图的线条大小（以像素为单位）。	1
原点 {x_pos y_pos}	将原点设置为笛卡尔 (x_pos, y_pos) 或当前 (x, y)（如果未指定值）。	0,0
设置 xy	将绘图位置设置为 (x,y) 。LINE 之后,端点将成为新的绘图位置。	
点 {线条大小 {不透明度}}	使用可选的 LINESIZE 和 OPACITY 覆盖在当前位置绘制一个点。	
线 xy {线尺寸 {不透明度}}	从当前位置到 (x,y) 画一条线,并使用可选的 LINESIZE 和 OPACITY 覆盖。	
圆圈直径 {线条大小 {不透明度}}	围绕当前位置绘制一个圆圈,线条大小可选（无/0 = 实线）并且不透明度覆盖。	
OVAL 宽度高度 {linesize {opacity}}	在当前位置周围绘制一个椭圆,线条大小可选（无/0 = 实线）并且不透明度覆盖。	
BOX 宽度高度 {线条大小 {不透明度}}	在当前位置周围绘制一个框,具有可选的线条大小（无/0 = 实线）和不透明度覆盖。	
OBOX 宽度 高度 x_radius y_radius {线条大小 {不透明度}}	在当前位置周围绘制一个圆角框,具有宽度、高度、x 和 y 半径以及可选的线条大小（无/0 = 实心）和 OPACITY 覆盖。	
TEXTSIZE尺寸	设置文本大小（6..200）。	10
TEXTSTYLE style_YYXXUIWW	将文本样式设置为 %YYXXUIWW: %YY 是垂直对齐:%00 = 中间,%10 = 底部,%11 = 顶部。	%00000001

	%XX 是水平对齐:%00 = 中间,%10 = 右,%11 = 左。 %U 是下划线:%1 = 下划线。 %l 是斜体: %1 = 斜体。 %WW 表示重量:%00 = 细,%01 = 正常,%10 = 粗体,%11 = 重。	
TEXTANGLE 角度	设置文本角度。在笛卡尔模式下,角度以度为单位。	0
文本 {大小 {样式 {角度}}} ‘文本’	绘制文本,并覆盖大小、样式和角度。要更改文本颜色,请在 TEXT 之前声明颜色。	
极地 {twopi {偏移}}	设置极坐标模式、twopi 值和偏移量。例如,POLAR -12 -3 就像一个钟面。 对于 twopi 值为 \$100000000 或 -\$100000000 时,使用 0 或 -1。 在极坐标模式下,(x, y) 坐标被解释为 (长度,角度)。	1亿美元,0
笛卡尔	设置笛卡尔模式。这是默认模式。	
清除	将绘图清除为背景颜色。	
更新	使用当前图更新窗口。用于 UPDATE 模式。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

* 颜色是模态值,否则为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,后跟可选的 0..15 亮度（默认为 8）。

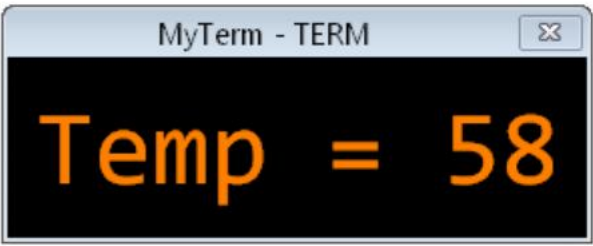
項目顯示

文本显示终端

```
CON_clkfreq = 10_000_000

PUB 去()我

    调试 (` TERM MyTerm SIZE 9 1 TEXTSIZE 40)
    重复
        重复 i 从 50 到 60
            调试 (` MyTerm 1  Temp = ` (i)  )
            等待毫秒(500)
```



TERM 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为 “字符串” 。	<无>
POS 左上角	设置窗口位置。	0,0
大小 列 行	设置终端列数（1..256)和终端行数（1..256）。	40, 20
TEXTSIZE尺寸	设置终端文本大小（6..200）。	编辑器文字大小
颜色 文本颜色 背景颜色 ...	设置文本颜色和背景颜色组合 #0..#3.*	默认颜色
背景颜色	设置显示背景颜色。	黑色的
更新	设置更新模式。只有输入 “更新”命令时,显示才会更新。	自动更新
学期喂养	描述	默认
特点	0 = 清除终端显示和主光标。 1 = 主光标。 2 = 将列设置为下一个字符值。 3 = 将行设置为下一个字符值。 4 = 选择颜色组合#0。 5 = 选择颜色组合#1。 6 = 选择颜色组合#2。 7 = 选择颜色组合#3。 8 = 退格键。 9 = 跳至下一个第 8 列。 13+10 或 13 或 10 = 新行。 32..255 = 可打印字符。	
细绳	打印字符串。	
清除	清除显示内容,改为背景色。	
更新	使用当前文本屏幕更新窗口。用于 UPDATE 模式。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

* 颜色是模态值,否则为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,后跟可选的 0..15 亮度（默认为 8）。

位图显示像素驱动的位图

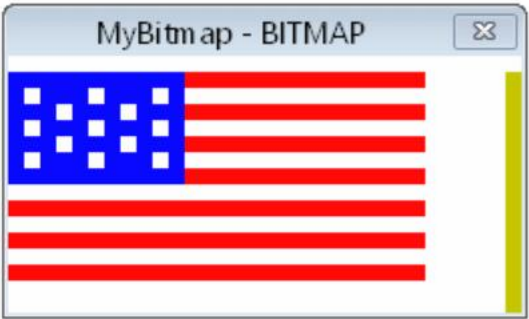
```
CON _clkfreq = 10_000_000

PUB 去()我

    调试 ( `位图 MyBitmap SIZE 32 16 DOTSIZE 8 LUT2 LONGS_2BIT)
    调试 ( `MyBitmap TRACE 14 LUTCOLORS 白色 红色 蓝色 黄色 6)
    重复
        调试 ( `MyBitmap`uhex_ (flag[i++ & $1F])`dly (100) )

数据保护

标志长 %%3333333333333330
    长整型 %%0010101022222220
    长%%0010101020202020
    长整型 %%001010101022222220
    长%%0010101022020220
    长整型 %%0010101022222220
    长%%0010101020202020
    长整型 %%0010101022222220
    长%%0010101022020220
    长整型 %%0010101022222220
    长%%0010101020202020
    长整型 %%0010101022222220
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0010101010101010
    长整型 %%0000000000000000
    长整型 %%0000000000000000
    长整型 %%0000000000000000
    长整型 %%0000000000000000
    长整型 %%0000000000000000
    长整型 %%0000000000000000
```



BITMAP 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
大小 x_像素 y_像素	设置位图中的像素数（x 和 y 均为 1..2048）。	256,256
DOTSIZE x_and_y_size {y_size}	设置显示像素的大小（1..256）。“DOTSIZE 16”使最终显示屏上的每个像素为 16x16。	1,1
lut1_to_rgb24	设置颜色模式。参见下图。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式,使用 RGB24 颜色加载 LUT。使用 HEX_LONG_ARRAY_ 进行加载。	默认颜色 0..7
跟踪 0_到_15	设置像素加载方向以及每行填满后是否滚动。见下方动画。	0
像素每次更新率	设置每次显示更新之前的像素数。“RATE -1”将速率设置为位图大小。	线尺寸
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<无>
更新	设置更新模式。只有输入“更新”命令时,显示才会更新。	自动更新
位图馈送	描述	默认
lut1_to_rgb24	更改颜色模式。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式,使用 rgb24 颜色加载 LUT。使用 HEX_LONG_ARRAY_ 加载颜色。	默认颜色 0..7
跟踪 0_到_15	更改像素加载到位图的方向。将速率设置为线条大小。	0
像素每次更新率	设置每次显示更新之前的像素数。“RATE -1”将速率设置为位图大小。	
设置 x_位置 {y_位置}	设置当前像素加载位置。通过清除 TRACE 的第 3 位取消滚动模式。	
滚动 x_scroll y_scroll	将位图滚动一定数量的像素。负值/正值决定方向,0 = 无。	
清除	将位图清除为零值像素。	
更新	使用当前位图更新窗口。用于 UPDATE 模式。	
保存 {WINDOW} ‘文件名’	保存整个窗口的位图文件 (.bmp) 或者仅保存 1 倍比例的位图。	
关闭	关闭窗口。	

TRACE 模式

速率设置为 1,以便每个像素在加载时都能被看到。

0

1

2

3

4

5

6

7

8

9

10

11

12

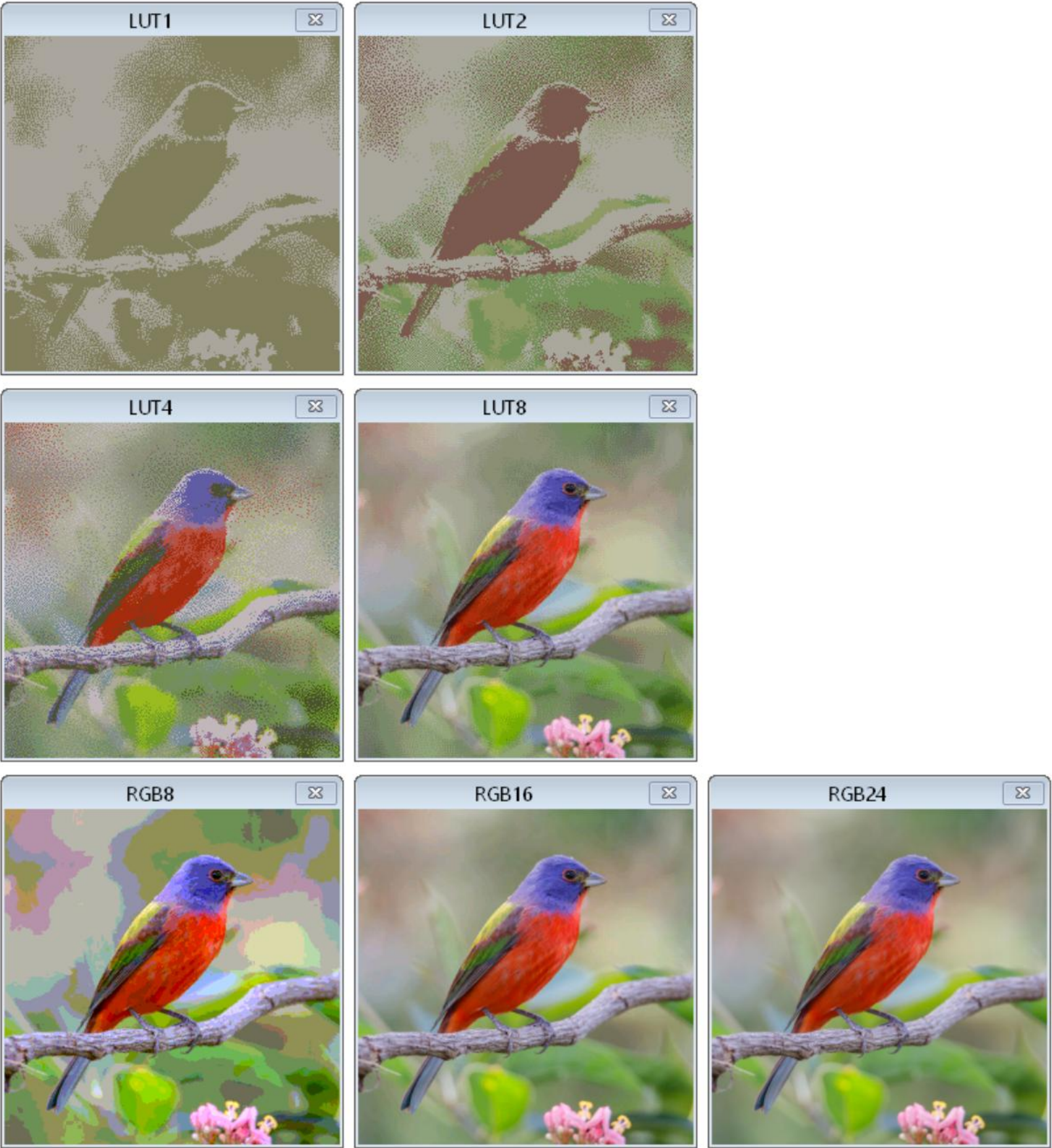
13

14

15

颜色模式	位/像素	描述	意图
查找表1	1	像素索引 LUT 颜色 0..1	节省内存的双色调板图形
查找表2	2	像素索引 LUT 颜色 0..3	节省内存的 4 色调板图形
LUT4	4	像素索引 LUT 颜色 0..15	节省内存的 16 色调板图形
LUT8	8	像素索引 LUT 颜色 0..255	节省内存的 256 色调板图形。
LUMA8	8	从黑色到彩色 *	亮度指示水平的仪器
LUMA8W	8	从白色到彩色 *	饱和度指示水平的仪器
LUMA8X	8	从黑色到彩色 * 变白	亮度指示水平的仪器,白色峰值
单细胞病毒8	8	从黑色到彩色:%HHHHSSSS	16 种色调,16 种亮度级别
HSV8W	8	从白色到彩色:%HHHHSSSS	16 种色调,16 种饱和度,源自白色
8型肝炎病毒	8	从黑色到彩色到白色:%HHHHSSSS	16 种色调,16 种亮度级别,白色达到峰值
RGBI8	8	从黑色到彩色:%RGBIII	8 种基本颜色,32 个亮度等级
RGBI8W	8	从白色到彩色:%RGBIII	8 种基本颜色,32 个饱和度等级,从白色开始
RGBI8X	8	从黑色到彩色到白色:%RGBIII	8 种基本颜色,32 个亮度等级,白色达到峰值
RGB8	8%RRRGGBB		字节级 RGB,具有 8 个红色,8 个绿色和 4 个蓝色级别
单细胞病毒16	16 从黑色到彩色:%HHHHHHHH_SS	SSSSSSS	256 种色调,256 种亮度级别
HSV16W	16 从白色到彩色:%HHHHHHHH_SS	SSSSSSS	256 种色调,256 种饱和度,源自白色
16型	16 从黑色到彩色再到白色:%HHHHHHHH_SS	SSSSSSS	256 种色调,256 种亮度级别,白色达到峰值
RGB16	16 %RRRRGGG_GGGBBBBB		具有 32 个红色级别,64 个绿色级别和 32 个蓝色级别的字级 RGB
RGB24	24 %RRRRRRR_GGGGGGG_BBBBBBB		全 RGB,红,绿,蓝 256 级

* 颜色为橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色。



CON_clkfreq = 100_000_000

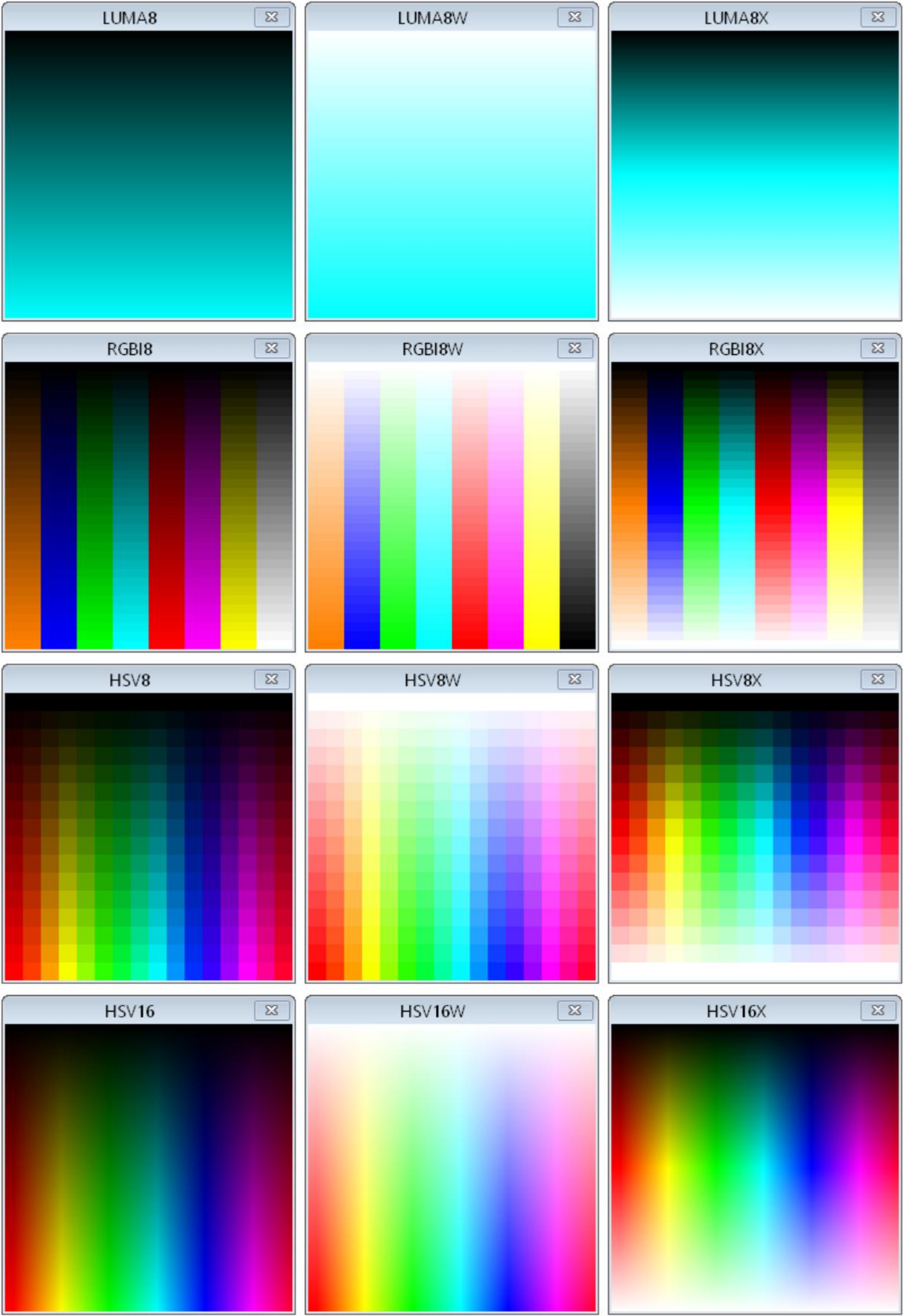
PUB 前往() | i 调试
(`位图 a 标题 LUT1 位置 100 100 跟踪 2 lut1 longs\_1bit alt) 调试 ( `位图 b 标题 LUT2 位置 370 100 跟踪 2 lut2 longs_2bit alt) 调试 (`位图 c 标题 LUT4 位置 100 395 跟踪 2 lut4 longs_4bit alt) 调试
(`位图 d 标题 LUT8 位置 370 395 跟踪 2 lut8 longs\_8bit) 调试 ( `位图 e 标题 RGB8 位置 100 690 跟踪 2 rgb8) 调试 (`位图 f 标题 RGB16 位置 370 690 跟踪 2 rgb16) 调试 ( `位图 g 标题 RGB24 位置 640 690 跟踪 2 rgb24)等待毫秒 (1000)

```
showbmp( a ,@image1,$8A,2,$800) showbmp( b , 发送 LUT1 图像 发
@image2,$36,4,$1000) showbmp( c ,@image3, 送 LUT2 图像 发送
$8A,16,$2000) showbmp( d ,@image4,$36,256, LUT4 图像 发送 LUT8
$4000) 图像

i:=@image5+$36 从同一个 24 bpp 文件发送 RGB8/RGB16/RGB24 图像 repeat $10000 debug(`e `uhex_(byte[i+0]>>6+byte[i+1]>>5<<2+
byte[i+2]>>5<<5))
debug(`f `uhex_(byte[i+0]>>3+byte[i+1]>>2<<5+byte[i+2]>>3<<11)) debug(`g `uhex_(byte[i+0]))+byte[i+1]<<8+byte[i+2]<<16 i+=3
```

PR! showbmp(字母,图像地址,lut偏移量,lut大小,图像长度) | i
image_address += lut_offset
debug(` `(letter) lutcolors `uhex_long_array_(image_address, lut_size)) image_address += lut_size <<
2 - 4 重复 image_longs debug(` `(letter)
`uhex_(long[image_address
+= 4]))

DAT
图像 1 文件 "bird_lut1.bmp" 图像 2 文件 "bird_lut2.bmp"
图像 3 文件 "bird_lut4.bmp" 图像 4 文件 "bird_lut8.bmp"
图像 5 文件 "bird_rgb24.bmp"



```
CON_clkfreq = 100_000_000

PUB 前往() | i 调试
    ( `位图 a 标题  LUMA8   位置 100 100 大小 1 256 点大小 256 1 luma8 青色)调试 ( `位图 b 标题  LUMA8W   位置 370 100 大小 1 256 点大小 256 1 luma8w 青色)调试 ( `位图 c 标题  LUMA8X   位置 640 100 大小 1 256 点大小 256 1 luma8x 青色)调试 ( `位图 d 标题  RGBI8   位置 100 395 大小 8 32 点大小 32 8 跟踪 4 rgbi8)调试 ( `位图 e 标题  RGBI8W   位置 370 395 大小 8 32 点大小 32 8 跟踪 4 rgbi8w)调试 ( `位图 f 标题  RGBI8X   位置 640 395 大小 8 32 点大小 32 8 跟踪 4 rgbi8x) 调试 ( `位图 g 标题  HSV8   位置 100 690 大小 16 16 跟踪 4 点大小 16 hsv8) 调试 ( `位图 h 标题  HSV8W   位置 370 690 大小 16 16 跟踪 4 点大小 16 hsv8w) 调试 ( `位图 i 标题  HSV8X   位置 640 690 大小 16 16 跟踪 4 点大小 16 hsv8x) 调试 ( `位图 j 标题  HSV16   位置 100 985 大小 256 256 跟踪 4 hsv16) 调试 ( `位图 k 标题  HSV16W   位置 370 985 大小 256 256 跟踪 4 hsv16w) 调试 ( `位图 l 标题  HSV16X   pos 640 985 大小 256 256 跟踪 4 hsv16x)waitms (1000)重复 i 从 0 到 255 调试 ( ` abcdefghi` uhex_ (i) )

    输入 8 位显示器

    重复 i 从 0 到 65535 debug(` jkl` uhex_(i))

    供给 16 位显示器
```


MIDI显示MIDI 键盘用于通过力度查看音符开启/关闭状态

CON_clkfreq = 10_000_000

PUB 去()我

调试 (`midi MyMidi 大小 3 范围 36 84)

重复

重复 i 从 36 到 84

调试 (`MyMidi \$90` (i,getrnd ()&\$7F))

等待毫秒(150)

调试 (`MyMidi \$80` (i,0))

MyMidi - MIDI

MIDI 实例	描述	默认
TITLE ‘字符串’	将窗口标题设置为“字符串”。	<无>
POS 左上角	设置窗口位置。	0,0
尺寸 键盘尺寸	设置 MIDI 键盘显示的大小 (1..50)。	4
范围 第一个键 最后一个键	设置第一个和最后一个 MIDI 键编号 (0..127)。	21,108 (88键)
频道 频道编号	设置要观察的 MIDI 通道号 (0..15)。	0
颜色 白键 黑键	设置白键和黑键的 ‘ON’ 颜色。 *	青色,洋红色
MIDI 喂食	描述	默认
字节	如果 \$90 + 通道,则为 NOTE_ON 模式,否则如果 \$80 + 通道,则为 NOTE_OFF 模式。 如果是 NOTE_ON 模式,则接收一个键 (\$00..\$7F)及其速度 (\$00..\$7F) ,更新显示。 如果是 NOTE_OFF 模式,则接收一个键 (\$00..\$7F)及其速度 (\$00..\$7F) ,更新显示。	
清除	清除所有注释。	
保存 {WINDOW} ‘文件名’	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

* 颜色为黑色 / 白色或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 洋红 / 黄色 / 灰色,亮度可选 0..15（默认为 8）。

这是一个 PASM 程序,它在 P16 上接收 MIDI 串行并将其发送到 MIDI 显示器:

反对

_clkfreq 接

= 10_000_000

收引脚

= 16

数据保护

组织

指令集

调试 (`midi m 大小 2)

wrpin #%11111_0,#rxpin

wxpin ##(clkfreq_/31250) << 16 + 8-1,#rxpin

drvl #rxpin

.wait testp #rxpin wc

if_nc jmp #.等待

rdpin x,#rxpin

shr x,#32-8

调试 (“`m” ,uhex_byte_ (x))

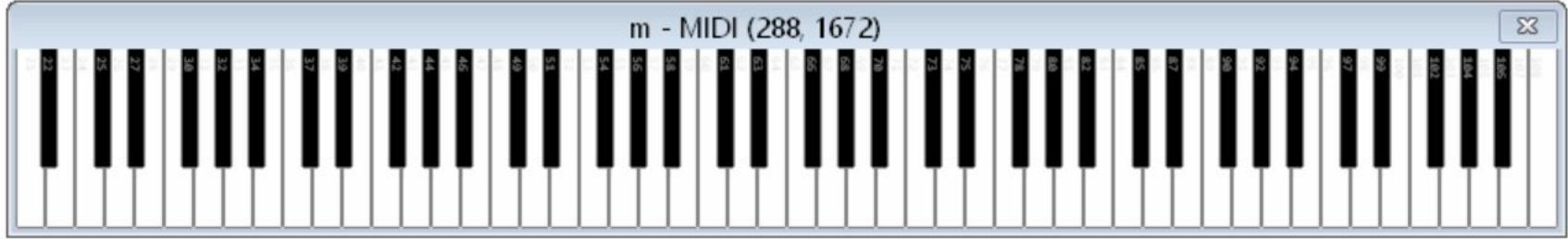
跳转

#.等待

+

水库

1



打包数据模式

打包数据模式用于高效传输字节数据类型,方法是让主机端从接收到的字节、字或长整型数据中解包。此外,字节可以在字内发送,长整型,并且可以在长整型内发送单词,以提高效率。

要建立打包数据操作,您必须指定下面列出的模式之一,后跟可选的“ALT”和“SIGNED”关键字:

打包模式 {ALT} {签名}

ALT关键字将导致每个发送字节中的位、双位或半字节在主机端重新排序。这简化了您发送的原始数据具有相对于您使用的 DEBUG 显示,其位字段的顺序是乱序的。对于以标准格式编写的位图数据,这很可能是需要的。

SIGNED关键字将导致所有解包数据值在主机端进行符号扩展。

打包数据 模式	描述	最终值	最终值 如果已签署
LONGS_1BIT	每个接收到的值都被转换成 32 个独立的 1 位值,从接收到的值的 LSB 开始。	0..1	-1..0
LONGS_2BIT	每个接收到的值都被转换成 16 个独立的 2 位值,从接收到的值的 LSB 开始。	0..3	-2..1
LONGS_4BIT	每个接收到的值都被转换成 8 个独立的 4 位值,从接收到的值的 LSB 开始。	0..15	-8..7
LONGS_8BIT	每个接收到的值都被转换成 4 个独立的 8 位值,从接收到的值的 LSB 开始。	0..255	-128..127
LONGS_16BIT	每个接收到的值都被转换成 2 个独立的 16 位值,从接收到的值的 LSB 开始。	0..65,535	-32,768..32,767
WORDS_1BIT	每个接收到的值都被转换成 16 个独立的 1 位值,从接收到的值的 LSB 开始。	0..1	-1..0
WORDS_2BIT	每个接收到的值都被转换成 8 个独立的 2 位值,从接收到的值的 LSB 开始。	0..3	-2..1
字_4位	每个接收到的值都被转换成 4 个独立的 4 位值,从接收到的值的 LSB 开始。	0..15	-8..7
字_8位	每个接收到的值都被转换成 2 个独立的 8 位值,从接收到的值的 LSB 开始。	0..255	-128..127
BYTES_1BIT	每个接收到的值都被转换成 8 个独立的 1 位值,从接收到的值的 LSB 开始。	0..1	-1..0
BYTES_2BIT	每个接收到的值都被转换成 4 个独立的 2 位值,从接收到的值的 LSB 开始。	0..3	-2..1
BYTES_4BIT	每个接收到的值都被转换成 2 个独立的 4 位值,从接收到的值的 LSB 开始。	0..15	-8..7

智能引脚配置的内置符号

智能引脚符号值	符号名称	细节
输入极性	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_TRUE_A (默认)	真 A 输入
%1000_0000_000_00000000000000_00_00000_0	反转	反转 A 输入
输入选择	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_LOCAL_A (默认)	选择 A 输入的本地引脚
%0001_0000_000_00000000000000_00_00000_0	附加组件	选择引脚 +1 作为 A 输入
%0010_0000_000_00000000000000_00_00000_0	加号2	选择引脚 +2 作为 A 输入
%0011_0000_000_00000000000000_00_00000_0	加3号	选择引脚+3 作为 A 输入
%0100_0000_000_00000000000000_00_00000_0	输出位	选择 OUT 位作为 A 输入
%0101_0000_000_00000000000000_00_00000_0	减3_A	选择引脚 3 作为 A 输入
%0110_0000_000_00000000000000_00_00000_0	减2_A	选择引脚 2 作为 A 输入
%0111_0000_000_00000000000000_00_00000_0	减1_A	选择引脚 1 作为 A 输入
B 输入极性	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_TRUE_B (默认)	真 B 输入
%0000_1000_000_00000000000000_00_00000_0	反转_B	反转 B 输入
B 输入选择	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_LOCAL_B (默认)	选择 B 输入的本地引脚
%0000_0001_000_00000000000000_00_00000_0	附加1	选择引脚 +1 作为 B 输入
%0000_0010_000_00000000000000_00_00000_0	附加2	选择引脚 +2 作为 B 输入
%0000_0011_000_00000000000000_00_00000_0	加3_B	选择引脚+3 作为 B 输入
%0000_0100_000_00000000000000_00_00000_0	输出位	选择 OUT 位作为 B 输入
%0000_0101_000_00000000000000_00_00000_0	减3_B	选择引脚 3 作为 B 输入
%0000_0110_000_00000000000000_00_00000_0	减2_B	选择引脚 2 作为 B 输入
%0000_0111_000_00000000000000_00_00000_0	最小 1_B	选择引脚 1 作为 B 输入
A,B输入逻辑	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_PASS_AB (默认)	选择 A,B
%0000_0000_001_00000000000000_00_00000_0	和 AB	选择 A & B,B
%0000_0000_010_00000000000000_00_00000_0	或 AB	选择 A B,B
%0000_0000_011_00000000000000_00_00000_0	异或运算	选择 A ^ B,B
%0000_0000_100_00000000000000_00_00000_0	保险丝	选择 A,B 的 FILT0 设置
%0000_0000_101_00000000000000_00_00000_0	过滤精度1	选择 A,B 的 FILT1 设置
%0000_0000_110_00000000000000_00_00000_0	过滤精度2	选择 A,B 的 FILT2 设置
%0000_0000_111_00000000000000_00_00000_0	过滤	选择 A,B 的 FILT3 设置
低电平引脚模式	(选一个)	
逻辑/施密特/比较器输入模式		
%0000_0000_000_00000000000000_00_00000_0	P_LOGIC_A (默认)	逻辑电平A→IN,输出OUT
%0000_0000_000_00010000000000_00_00000_0	逻辑A_FB	逻辑电平A→IN,输出反馈
%0000_0000_000_00100000000000_00_00000_0	逻辑B	逻辑电平B→IN,输出反馈
%0000_0000_000_00110000000000_00_00000_0	施密特	施密特触发器A→IN,输出OUT
%0000_0000_000_01000000000000_00_00000_0	施密特_A_FB	施密特触发器A→IN,输出反馈
%0000_0000_000_01010000000000_00_00000_0	施密特_B_FB	施密特触发器B→IN,输出反馈
%0000_0000_000_01100000000000_00_00000_0	比较器	A > B → IN,输出OUT
%0000_0000_000_01110000000000_00_00000_0	比较器	A > B → IN,输出反馈
%xxxx_xxxx_xxx_xxxxSIOHHLLLL_xx_xxxxx_x		同步模式、输入/输出极性、高/低驱动
ADC 输入模式		
%0000_0000_000_10000000000000_00_00000_0	ADC 接口	ADC GIO → IN,输出OUT
%0000_0000_000_10000100000000_00_00000_0	电压输入	ADC VIO → IN,输出OUT

%0000_0000_000_1000100000000_00_00000_0	ADC 浮点数	ADC FLOAT → IN,输出OUT
%0000_0000_000_1000110000000_00_00000_0	ADC_1X接口	ADC 1x → 输入,输出 OUT
%0000_0000_000_1001000000000_00_00000_0	ADC3X接口	ADC 3.16x → 输入,输出 OUT
%0000_0000_000_1001010000000_00_00000_0	电压模数转换器	ADC 10x → 输入,输出 OUT
%0000_0000_000_1001100000000_00_00000_0	ADC_30X 电源	ADC 31.6x → 输入,输出 OUT
%0000_0000_000_1001110000000_00_00000_0	ADC_100X 电源	ADC 100x → 输入,输出 OUT
%xxxx_xxxx_xxx_xxxxxxOHHHLLL_xx_xxxxx_x		O = 输出极性,HHH/LLL = 高/低驱动
DAC 输出模式		DIR 使能输出,OUT 使能 ADC
%0000_0000_000_1010000000000_00_00000_0	DAC_990R_3V 电源	DAC 990Ω,3.3V 峰值,ADC 1x → IN
%0000_0000_000_1010100000000_00_00000_0	DAC_600R_2V	DAC 600Ω,2.0V 峰值,ADC 1x → IN
%0000_0000_000_1011000000000_00_00000_0	DAC_124R_3V 电源	DAC 123.75Ω,3.3V 峰值,ADC 1x → IN
%0000_0000_000_1011100000000_00_00000_0	DAC_75R_2V 电源	DAC 75Ω,2.0V 峰值,ADC 1x → IN
%xxxx_xxxx_xxx_xxxxxDDDDDDDD_xx_xxxxx_x		DDDDDDDD = 8 位 DAC 值
级别比较模式		DIR 使能输出 (1.5kΩ 驱动)
%0000_0000_000_1100000000000_00_00000_0	级别 A	A > 电平 → IN,输出 OUT
%0000_0000_000_1101000000000_00_00000_0	级别 A_FBN	A>Level→IN,输出负反馈
%0000_0000_000_1110000000000_00_00000_0	级别 B_FBP	B>电平→IN,输出正反馈
%0000_0000_000_1111000000000_00_00000_0	级别 B_FBN	B>Level→IN,输出负反馈
%xxxx_xxxx_xxx_xxxxSLLLLLLLL_xx_xxxxx_x		S = 同步,LLLLLLLL = 8 位级别
低级引脚子模式		
同步模式	(选一个)	(适用于逻辑/施密特/比较器/电平模式)
%xxxx_xxxx_xxx_xxxxSxxxxxxxx_xx_xxxxx_x		同步模式位
%0000_0000_000_0000000000000_00_00000_0	P_ASYNC_IO (默认)	选择异步 I/O
%0000_0000_000_0000100000000_00_00000_0	同步输入输出	选择同步 I/O
输入极性	(选一个)	(适用于逻辑/施密特/比较器模式)
%xxxx_xxxx_xxx_xxxxIxxxxxxx_xx_xxxxx_x		IN 极性位
%0000_0000_000_0000000000000_00_00000_0	P_TRUE_IN (默认)	真 IN 位
%0000_0000_000_0000010000000_00_00000_0	反转输入	反转 IN 位
输出极性	(选一个)	(适用于逻辑/施密特/比较器/ADC 模式)
%xxxx_xxxx_xxx_xxxxOxxxxxx_xx_xxxxx_x		输出极性位
%0000_0000_000_0000000000000_00_00000_0	P_TRUE_OUTPUT (默认)	选择真实输出
%0000_0000_000_0000001000000_00_00000_0	反转输出	选择反相输出
驱动-高强度	(选一个)	(适用于逻辑/施密特/比较器/ADC 模式)
%xxxx_xxxx_xxx_xxxxxxHHHxx_xx_xxxxx_x		驱动高选择器位
%0000_0000_000_0000000000000_00_00000_0	P_HIGH_FAST (默认)	快速驱动高电平 (30mA)
%0000_0000_000_0000000001000_00_00000_0	高_1K5	驱动高1.5kΩ
%0000_0000_000_0000000010000_00_00000_0	上限_15K	驱动高15kΩ
%0000_0000_000_0000000011000_00_00000_0	最高温度	驱动高150kΩ
%0000_0000_000_0000000100000_00_00000_0	最高价	驱动高1mA
%0000_0000_000_0000000101000_00_00000_0	上限为 100UA	驱动高 100μA
%0000_0000_000_0000000110000_00_00000_0	上限	驱动高 10μA
%0000_0000_000_0000000111000_00_00000_0	浮动	浮高
低强度驱动	(选一个)	(适用于逻辑/施密特/比较器/ADC 模式)
%xxxx_xxxx_xxx_xxxxxxxxxxLLL_xx_xxxxx_x		驱动低选择器位
%0000_0000_000_0000000000000_00_00000_0	P_LOW_FAST (默认)	快速驱动至低电平 (30mA)
%0000_0000_000_00000000000001_00_00000_0	低_1K5	驱动低1.5kΩ
%0000_0000_000_00000000000010_00_00000_0	低电平	驱动低 15kΩ
%0000_0000_000_00000000000011_00_00000_0	低电压 150K	驱动低 150kΩ
%0000_0000_000_0000000000100_00_00000_0	低电平	驱动低 1mA
%0000_0000_000_0000000000101_00_00000_0	低_100UA	驱动低 100μA
%0000_0000_000_0000000000110_00_00000_0	低电平	驱动低 10μA

%0000_0000_000_00000000000111_00_00000_0	低浮点数	低浮力
DIR/OUT 控制	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_TT_00 (默认)	TT = %00
%0000_0000_000_00000000000000_01_00000_0	P_TT_01	TT = %01
%0000_0000_000_00000000000000_10_00000_0	P_TT_10	TT = %10
%0000_0000_000_00000000000000_11_00000_0	P_TT_11	TT = %11
%0000_0000_000_00000000000000_01_00000_0	电源	在智能引脚模式下启用输出
%0000_0000_000_00000000000000_01_00000_0	通道	在非智能引脚 DAC 模式下启用 DAC 通道
%0000_0000_000_00000000000000_10_00000_0	比特数转换器	启用 BITDAC 以实现非智能引脚 DAC 模式
智能密码模式	(选一个)	
%0000_0000_000_00000000000000_00_00000_0	P_NORMAL (默认)	正常模式 (非智能密码模式)
%0000_0000_000_00000000000000_00_00001_0	存储库	长存储库 (非 DAC 模式)
%0000_0000_000_00000000000000_00_00001_0	DAC噪音	DAC 噪声 (DAC 模式)
%0000_0000_000_00000000000000_00_00010_0	DAC抖动RND	DAC 16 位随机抖动 (DAC 模式)
%0000_0000_000_00000000000000_00_00011_0	DAC_DITHER_PWM	DAC 16 位 PWM 抖动 (DAC 模式)
%0000_0000_000_00000000000000_00_00100_0	脉冲	脉冲/周期输出
%0000_0000_000_00000000000000_00_00101_0	过渡	过渡输出
%0000_0000_000_00000000000000_00_00110_0	频率	NCO 频率输出
%0000_0000_000_00000000000000_00_00111_0	军士职责	NCO 职责输出
%0000_0000_000_00000000000000_00_01000_0	脉冲宽度调制三角形	PWM 三角波输出
%0000_0000_000_00000000000000_00_01001_0	脉冲宽度调制锯齿波	PWM锯齿波输出
%0000_0000_000_00000000000000_00_01010_0	PWM 开关电源	PWM 开关电源 I/O
%0000_0000_000_00000000000000_00_01011_0	P_QUADRATURE	AB正交编码器输入
%0000_0000_000_00000000000000_00_01100_0	寄存器	B 高时 A 上升,B 下降
%0000_0000_000_00000000000000_00_01101_0	寄存器向上向下	当 B 高时,A 上升时增加,当 B 低时,A 上升时减少
%0000_0000_000_00000000000000_00_01110_0	上升次数	在 A 点上升时增加,在 B 点上升时可选下降
%0000_0000_000_00000000000000_00_01111_0	最高分数	在 A 高点时增加,在 B 高点时可选地减少
%0000_0000_000_00000000000000_00_10000_0	状态检测	对于 A-low 和 A-high 状态,计数刻度
%0000_0000_000_00000000000000_00_10001_0	最高价	对于 A 高状态,计数刻度
%0000_0000_000_00000000000000_00_10010_0	事件触发	对于 X A 高点/上升/边缘,计数刻度/ 无 A 高/上升/边缘的 X 刻超时
%0000_0000_000_00000000000000_00_10011_0	周期计时	对于 A 的 X 个周期,计数刻度
%0000_0000_000_00000000000000_00_10100_0	周期高点	对于 A 的 X 个周期,计算最高点
%0000_0000_000_00000000000000_00_10101_0	计数标记	对于 X+ 个刻度内的 A 周期,计数刻度
%0000_0000_000_00000000000000_00_10110_0	最高点数	对于 X+ 个刻度内的 A 周期,计算高点
%0000_0000_000_00000000000000_00_10111_0	计数周期	对于 X+ 个刻度内的 A 周期,计数周期
%0000_0000_000_00000000000000_00_11000_0	压降模数转换器	ADC 采样/过滤/捕获,内部时钟
%0000_0000_000_00000000000000_00_11001_0	ADC 输出	ADC 采样/过滤/捕获,外部时钟
%0000_0000_000_00000000000000_00_11010_0	ADC 范围	带触发器的 ADC 示波器
%0000_0000_000_00000000000000_00_11011_0	USB 对	USB 引脚对
%0000_0000_000_00000000000000_00_11100_0	同步发送	同步串行传输
%0000_0000_000_00000000000000_00_11101_0	同步接收	同步串行接收
%0000_0000_000_00000000000000_00_11110_0	异步发送	异步串行传输
%0000_0000_000_00000000000000_00_11111_0	异步接收	异步串行接收

内置 Streamer 模式符号

飘带符号值	符号名称
#立即 → LUT → 引脚 / DAC	
%0000_0000_0000_0000 << 16 %0000_DDDD_EPPP_BBBB << 16	修改器
%0001_0000_0000_0000 << 16 %0001_DDDD_EPPP_BBBB << 16	16X2_LUT 版本
%0010_0000_0000_0000 << 16 %0010_DDDD_EPPP_BBBB << 16	8X4_LUT
%0011_0000_0000_0000 << 16 %0011_DDDD_EPPP_BBBB << 16	4X8_LUT
#立即 → 引脚 / DAC	
%0100_0000_0000_0000 << 16 %0100_DDDD_EPPP_PPPA << 16	X_IMM_32X1_1DAC1
%0101_0000_0000_0000 << 16 %0101_DDDD_EPPP_PP0A << 16	X_IMM_16X2_2DAC1
%0101_0000_0000_0010 << 16 %0101_DDDD_EPPP_PP1A << 16	X_IMM_16X2_1DAC2
%0110_0000_0000_0000 << 16 %0110_DDDD_EPPP_P00A << 16	X_IMM_8X4_4DAC1
%0110_0000_0000_0010 << 16 %0110_DDDD_EPPP_P01A << 16	8X4_2DAC2 是
%0110_0000_0000_0100 << 16 %0110_DDDD_EPPP_P10A << 16	X_IMM_8X4_1DAC4
%0110_0000_0000_0110 << 16 %0110_DDDD_EPPP_0110 << 16	4X8_4DAC2
%0110_0000_0000_0111 << 16 %0110_DDDD_EPPP_0111 << 16	X_IMM_4X8_2DAC4
%0110_0000_0000_1110 << 16 %0110_DDDD_EPPP_1110 << 16	X_IMM_4X8_1DAC8
%0110_0000_0000_1111 << 16 %0110_DDDD_EPPP_1111 << 16	X_IMM_2X16_4DAC4
%0111_0000_0000_0000 << 16 %0111_DDDD_EPPP_0000 << 16	X_IMM_2X16_2DAC8
%0111_0000_0000_0001 << 16 %0111_DDDD_EPPP_0001 << 16	X_IMM_1X32_4DAC8
RDFAST → LUT → 引脚/DAC	
%0111_0000_0000_0010 << 16 %0111_DDDD_EPPP_001A << 16	修改器
%0111_0000_0000_0100 << 16 %0111_DDDD_EPPP_010A << 16	16X2_LUT 版本
%0111_0000_0000_0110 << 16 %0111_DDDD_EPPP_011A << 16	查询结果为 X_RFLONG_8X4_LUT
%0111_0000_0000_1000 << 16 %0111_DDDD_EPPP_1000 << 16	4X8_LUT
RDFAST → 引脚 / DAC	
%1000_0000_0000_0000 << 16 %1000_DDDD_EPPP_PPPA << 16	X_RFBYTE_1P_1DAC1
%1001_0000_0000_0000 << 16 %1001_DDDD_EPPP_PP0A << 16	X_RFBYTE_2P_2DAC1
%1001_0000_0000_0010 << 16 %1001_DDDD_EPPP_PP1A << 16	X_RFBYTE_2P_1DAC2
%1010_0000_0000_0000 << 16 %1010_DDDD_EPPP_P00A << 16	X_RFBYTE_4P_4DAC1
%1010_0000_0000_0010 << 16 %1010_DDDD_EPPP_P01A << 16	射频字节_4P_2DAC2
%1010_0000_0000_0100 << 16 %1010_DDDD_EPPP_P10A << 16	X_RFBYTE_4P_1DAC4
%1010_0000_0000_0110 << 16 %1010_DDDD_EPPP_0110 << 16	暂无说明,留下第一条!
%1010_0000_0000_0111 << 16 %1010_DDDD_EPPP_0111 << 16	射频字节4
%1010_0000_0000_1110 << 16 %1010_DDDD_EPPP_1110 << 16	X_RFBYTE_8P_1DAC8
%1010_0000_0000_1111 << 16	射频字_16P_4DAC4

%1010_DDDD_EPPP_1111 << 16	
%1011_0000_0000_0000 << 16 %1011_DDDD_EPPP_0000 << 16	射频字_16P_2DAC8
%1011_0000_0000_0001 << 16 %1011_DDDD_EPPP_0001 << 16	暂无说明,留下第一条!
RDFAST → RGB → 引脚 / DAC	
%1011_0000_0000_0010 << 16 %1011_DDDD_EPPP_0010 << 16	暂无说明,留下第一条!
%1011_0000_0000_0011 << 16 %1011_DDDD_EPPP_0011 << 16	RGBI8 参数
%1011_0000_0000_0100 << 16 %1011_DDDD_EPPP_0100 << 16	RGB8 参数
%1011_0000_0000_0101 << 16 %1011_DDDD_EPPP_0101 << 16	参数
%1011_0000_0000_0110 << 16 %1011_DDDD_EPPP_0110 << 16	X_RFLONG_RGB24
引脚 → DAC / WRFAST	
%1100_0000_0000_0000 << 16 %1100_DDDD_WPPP_PPPA << 16	X_1P_1DAC1_WFBYTE
%1101_0000_0000_0000 << 16 %1101_DDDD_WPPP_PPOA << 16	X_2P_2DAC1_WFBYTE
%1101_0000_0000_0010 << 16 %1101_DDDD_WPPP_PP1A << 16	X_2P_1DAC2_WFBYTE
%1110_0000_0000_0000 << 16 %1110_DDDD_WPPP_P00A << 16	X_4P_4DAC1_WFBYTE
%1110_0000_0000_0010 << 16 %1110_DDDD_WPPP_P01A << 16	X_4P_2DAC2_WFBYTE
%1110_0000_0000_0100 << 16 %1110_DDDD_WPPP_P10A << 16	X_4P_1DAC4_WFBYTE
%1110_0000_0000_0110 << 16 %1110_DDDD_WPPP_0110 << 16	X_8P_4DAC2_WFBYTE
%1110_0000_0000_0111 << 16 %1110_DDDD_WPPP_0111 << 16	X_8P_2DAC4_WFBYTE
%1110_0000_0000_1110 << 16 %1110_DDDD_WPPP_1110 << 16	X_8P_1DAC8_WFBYTE
%1110_0000_0000_1111 << 16 %1110_DDDD_WPPP_1111 << 16	X_16P_4DAC4_WFWORD
%1111_0000_0000_0000 << 16 %1111_DDDD_WPPP_0000 << 16	X_16P_2DAC8_WFWORD
%1111_0000_0000_0001 << 16 %1111_DDDD_WPPP_0001 << 16	X_32P_4DAC8_WFLONG
ADC / 引脚 → DAC / WRFAST	
%1111_0000_0000_0010 << 16 %1111_DDDD_W000_0010 << 16	X_1ADC8_0P_1DAC8_WFBYTE
%1111_0000_0000_0011 << 16 %1111_DDDD_WPPP_0011 << 16	X_1ADC8_8P_2DAC8_WFWORD
%1111_0000_0000_0100 << 16 %1111_DDDD_W000_0100 << 16	X_2ADC8_0P_2DAC8_WFWORD
%1111_0000_0000_0101 << 16 %1111_DDDD_WPPP_0101 << 16	X_2ADC8_16P_4DAC8_WFLONG
%1111_0000_0000_0110 << 16 %1111_DDDD_W000_0110 << 16	X_4ADC8_0P_4DAC8_WFLONG
DDS/戈策尔	
%1111_0000_0000_0111 << 16 %1111_DDDD_0PPP_P111 << 16	X_DDS_GOERTZEL_SINC1
%1111_0000_1000_0111 << 16 %1111_DDDD_1PPP_P111 << 16	修改器
子字段	
DAC 通道输出	
%xxxx_DDDD_xxxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0001_0000_0000 << 16 %0000_0010_0000_0000 << 16 %0000_0011_0000_0000 << 16 %0000_0100_0000_0000 << 16 %0000_0101_0000_0000 << 16 %0000_0110_0000_0000 << 16 %0000_0111_0000_0000 << 16 %0000_1000_0000_0000 << 16 %0000_1001_0000_0000 << 16	X_DACS_OFF (默认) X_DACS_0_0_0_0 X_DACS_X_X_0_0 暂无数据 X_DACS_X_X_X_0 X_DACS_X_X_0_X 暂无数据 暂无数据 X_DACS_0N0_0N0 X_DACS_X_X_0N0

%0000_1010_0000_0000 << 16 %0000_1011_0000_0000 << 16 %0000_1100_0000_0000 << 16 %0000_1101_0000_0000 << 16 %0000_1110_0000_0000 << 16 %0000_1111_0000_0000 << 16	暂无数据 X_DACS_1_0_1_0 暂无数据 暂无数据 X_DACS_1N1_0N0 暂无
引脚输出控制	
%xxx_xxxx_Exxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0000_1000_0000 << 16	X_PINS_OFF (默认) X_PINS_ON
写入控制	
%xxxx_xxxx_Wxxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0000_1000_0000 << 16	X_WRITE_OFF (默认) 写入
1/2/4 位交替顺序	
%xxxx_xxxx_xxxx_xxxA << 16 %0000_0000_0000_0000 << 16 %0000_0000_0000_0001 << 16	X_ALT_OFF (默认) 开启

事件和中断源的内置符号

符号 值	符号名称
0	事件_INT / INT_OFF
1	事件_CT1
2	事件_CT2
3	事件_CT3
4	事件_SE1
5	事件_SE2
6	事件_SE3
7	事件_SE4
8	事件进程
9	事件_FBW
10	事件发送
11	事件_XFI
12	事件_XRO
十三	事件_XRL
14	事件_ATN
15	事件_QMT

用于 COGINIT 的内置符号

COGINIT 符号值	符号名称	细节
%00_0000	COGEXEC （默认）	使用 “COGEXEC + CogNumber”以 cogexec 模式启动 cog
%10_0000	执行程序	使用 “HUBEXEC + CogNumber”以 hubexec 模式启动 cog
%01_0000	COGEXEC_新	在 cogexec 模式下启动可用的 cog
%11_0000	HUBEXEC_NEW	在 hubexec 模式下启动可用的 cog
%01_0001	更新对	在 cogexec 模式下启动可用的 eve/odd 齿轮对,这对于 LUT 共享很有用
%11_0001	HUBEXEC_NEW_PAIR	在 hubexec 模式下启动可用的 eve/odd 齿轮对,这对于 LUT 共享很有用

内置 COGSPIN 用法符号

COGINIT 符号值	符号名称	细节
%01_0000	新冠疫情	启动可用的齿轮

内置数字符号

符号 值	符号名称	细节
\$0000_0000	错误的	与 0 相同
\$FFFF_FFFF	真的	与 -1 相同
\$8000_0000	负相关	负极端整数，-2_147_483_648 (\$8000_0000)
\$7FFF_FFFF	POSX	正极整数,+2_147_483_647 (\$7FFF_FFFF)
\$4049_0FDB	PI	圆周率的单精度浮点值,3.14159265

PNut.exe 的命令行选项

命令	行动	之后出现ERROR.TXT文件 (文件将包含以下其中一行)
普努特	启动 PNut.exe。	好的
pnut 文件名	加载文件名 (假定扩展名为 .spin2,但不强制执行) 。	好的
pnut 文件名 -c	加载并编译文件名,然后退出。	好的 <文件名路径>:<行号>:错误:<错误消息>
pnut 文件名 -r	加载、编译并运行文件名,然后退出。	好的 <文件名路径>:<行号>:错误:<错误消息> 串行错误
pnut 文件名 -rd	加载、编译、运行和调试文件名,然后在调试窗口退出 被关闭。	好的 <文件名路径>:<行号>:错误:<错误消息> 串行错误
pnut 文件名 -f	加载、编译和编程闪存文件名,然后退出。	好的 <文件名路径>:<行号>:错误:<错误消息> 串行错误
pnut 文件名 -fd	加载、编译、编程闪存和调试文件名,然后退出 调试窗口关闭。	好的 <文件名路径>:<行号>:错误:<错误消息> 串行错误
pnut -debug {CommPort} {BaudRate}以 BaudRate (默认值 = 2_000_000)打开 CommPort (默认值 = 1)并 呈现传入的 DEBUG 数据并显示。		好的 串行错误

包含批处理文件以调用 PNut.exe 并将状态返回到 STDOUT、STDERR 和 ERRORLEVEL

PNUT_SHELL.BAT 文件	批处理文件行说明
@echo 关闭 设置 ERROR_FILE=error.txt 如果存在 %ERROR_FILE% del /q /f %ERROR_FILE% 如果存在%1设置GOOD_SRC=1 如果存在 %1.spin2 设置 GOOD_SRC=1 如果定义了 GOOD_SRC (pnut_v35L %1 %2 %3 设置 pnuterror = %ERRORLEVEL% 对于/f “tokens=*”%%i 在 (%ERROR_FILE%)中执行 echo %%i 1>&2) 别的 (设置 pnuterror=-1 echo 错误:未找到文件 - %1 1>&2) 退出 %pnuterror%	取消回显到控制台。 设置 ERROR.TXT 文件名。 如果存在ERROR.TXT,请将其删除。 检查第一个参数是否为有效的源文件。 检查第一个参数是否为有效的 .spin2 源文件。 如果源文件存在 ...使用传递的参数调用 PNut。示例:pnut_shell filename -r ...从 PNut 捕获 ERRORLEVEL (0 = 正常,1 = 错误) 。 ...将 ERROR.TXT 文件复制到 STDOUT 和 STDERR。 别的 ...设置文件未找到错误。 ...将文件未找到错误消息返回到 STDOUT 和 STDERR。 返回 ERRORLEVEL。更改为 “exit /b %pnuterror%”以维护控制台窗口。

时钟设置

要为程序建立初始时钟设置,您可以声明某些符号,编译器将查找这些符号来确定您的设置。这些符号必须在以下某个位置定义:
以下组合:

CON 声明 (例如数字可能会有所不同)	影响	集线器 %CC_SS**
CON _clkfreq = 250_000_000 _errfreq = 0	选择 XI/XO-crystal-plus-PLL 模式,假定晶振为 20MHz。 将计算最佳 PLL 设置来实现 _clkfreq。 如果 _clkfreq ± _errfreq 无法实现,则编译失败。	10_11
CON _xtlfreq = 12_000_000 _clkfreq = 148_500_000 _errfreq = 150_000	选择 XI/XO-crystal-plus-PLL 模式以及频率。 将计算最佳 PLL 设置来实现 _clkfreq。 如果 _clkfreq ± _errfreq 无法实现,则编译失败。	1x_11
CON _xinfreq = 32_000_000 _clkfreq = 297_500_000 _errfreq = 100_000	选择 XI 输入加 PLL 模式以及频率。 将计算最佳 PLL 设置来实现 _clkfreq。 如果 _clkfreq ± _errfreq 无法实现,则编译失败。	01_10
CON _xtlfreq = 16_000_000	选择 XI/XO 晶体模式和频率。	1x_10
CON _xinfreq = 100_000_000	选择 XI 输入模式和频率。	01_10
连接 _rcslow	选择以~20KHz 运行的内部 RCSLOW 振荡器。	00_01
CON _rcfast	选择以 20MHz+ 运行的内部 RCFAST 振荡器。 如果未指定任何内容,这是默认模式。	00_00

* _errfreq 声明是可选的,因为 _errfreq 默认为 1_000_000。
** 如果 _xtlfreq >= 16_000_000 则 x=0 表示每个 XI/XO 为 15pF,否则 x=1 表示每个 XI/XO 为 30pF。

在编译期间,编译器定义了两个常量符号,其值反映了编译后的时钟设置:

象征	描述
时钟模式_	编译的时钟模式,可通过 HUBSET 设置。 · 对于 Spin2 程序,在程序启动之前,HUBSET 将使用 “clkmode_”调用,以设置编译时钟模式。“clkmode_”值也将存储在集线器中变量 ‘clkmode’ 。 · 对于纯 PASM 程序,可以使用 clkmode_ 将时钟模式设置为不同于其初始 RCFAST 设置可设置为任意晶振/PLL 编译设置,如下所示: HUBSET ##clkmode_ & !3 启动晶振/PLL,保持在 RCFAST WAITX ##20_000_000/100 等待 10 毫秒 HUBSET ##clkmode_ 切换到晶振/PLL · clkmode_ 值可能在应用程序层次结构的每个文件中有所不同。顶层以下的文件文件不会继承顶级文件的值。
时钟频率	编译的时钟频率。 · 对于 Spin2 程序,“clkfreq_”值将存储在集线器变量 “clkfreq”中。 · 对于纯 PASM 程序,“clkfreq_”只能作为常量引用。 · clkfreq_ 值可能在应用程序层次结构的每个文件中有所不同。顶层以下的文件文件不会继承顶级文件的值。

对于 Spin2 程序,维护两个反映当前时钟设置的集线器变量:

Spin2 变量	描述
时钟模式	当前时钟模式,位于 LONG[\$40]。使用 clkmode_ 值初始化。
时钟频率	当前时钟频率,位于 LONG[\$44]。使用 clkfreq_ 值初始化。
	 · 对于 Spin2 方法,这些变量可以作为 “clkmode”和 “clkfreq”读取和写入。 与直接写入这些变量相比,使用以下方法更安全: CLKSET (新时钟模式,新时钟频率) 这样,所有其他代码都会看到 “clkmode”和 “clkfreq”的快速、并行更新,并且时钟模式转换是安全地完成的,采用先前的值,以避免潜在的时钟故障。 · 对于在Spin2 下运行的PASM 代码,可以按如下方式读取和写入这些变量: RDLONG x,#@clkmode 将 clkmode 读入 x WRLONG x,#@clkmode 将 x 写入 clkmode RDLONG x,#@clkfreq 将 clkfreq 读入 x WRLONG x,#@clkfreq 将 x 写入 clkfreq 集合地点 #2-1 RDLONG x,#@clkmode 将 clkmode 和 clkfreq 读入 x 和 x+1 集合地点 #2-1 WRLONG x,#@clkmode 将 x 和 x+1 写入 clkmode 和 clkfreq

对于仅支持 PASM 的程序,有一条名为 ASMCLK 的特殊指令,它将设置时钟设置符号指定的时钟模式。ASMCLK 没有操作数,但可以与条件前缀。ASMCLK 将汇编为一条或六条 PASM 指令,具体取决于时钟模式。此指令旨在在仅 PASM 程序的启动时使用一次,假设当前选择了从启动时继承的 RCFast 模式:

CON 声明 (例如数字可能会有所不同)	集线器 %CC_SS	ASMCLK 汇编为:	
CON _clkfreq = 250_000_000 _errfreq = 0	10_11	集线器 ##clkmode_ & !%11 WAITX ##20_000_000/100 集线器 ##clkmode_ 启动外部时钟,保持在 RCFast 模式 让外部时钟稳定 10ms 切换到外部时钟模式	
CON _xtlfreq = 12_000_000 _clkfreq = 148_500_000 _errfreq = 150_000	1x_11		
CON _xinfreq = 32_000_000 _clkfreq = 297_500_000 _errfreq = 100_000	01_10		
CON _xtlfreq = 16_000_000	1x_10		
CON _xinfreq = 100_000_000	01_10		
连接 _rcslow	00_01	轮毂 #1	切换到 RCSLOW 模式
CON _rcfast	00_00	轮毂 #0	保持 RCFast 模式